

EEE 304 (July 2025)
Digital Electronics Laboratory

Section: B1 Group: 05

FPGA-BASED PLC (Programmable Logic
Controller)

Course Instructors:

Sadman Sakib Ahbab, Assistant Professor, Department of EEE, BUET
Archishman Sarkar, Part-Time Lecturer, Department of EEE, BUET

Signature of Instructor: _____

Academic Honesty Statement:

IMPORTANT! Please carefully read and sign the Academic Honesty Statement, below. Type the student ID and name, and put your signature. You will not receive credit for this project experiment unless this statement is signed in the presence of your lab instructor.

Signature: _____ Full Name: Tahmid Mahmud Fahim Student ID: 2106078	Signature: _____ Full Name: Abid Uz Zaman Student ID: 2106080
Signature: _____ Full Name: Robayet Hossen Bapon Student ID: 2106083	Signature: _____ Full Name: Md. Rakibul Islam Student ID: 2106084
Signature: _____ Full Name: Afrina Rahman Student ID: 2106085	

Table of Contents

1. Abstract.....	1
2. Introduction.....	1
3. Design	2
3.1. Problem Formulation (PO(b)).....	2
3.1.1. Identification of Scope	2
3.1.2. Literature Review.....	3
3.1.3. Formulation of Problem.....	3
3.1.4. Analysis	4
3.2. Design Method (PO(a)).....	4
3.2.1. Internal Program Representation	5
3.2.2. High-Level Block Diagram (Data Path)	6
3.2.3. Scan-Cycle Algorithm	6
3.2.4. Timer and Counter Algorithms	7
3.2.5. HDL Strategy	7
3.2.6. Simulation and Verification Strategy.....	7
3.2.7. Hardware Design and I/O Mapping.....	8
3.3. Circuit Diagram (Simulated on Xilinx Vivado).....	11
3.4. Full Source Code of Firmware 14	
3.4.1. Top Module.....	14
3.4.2. Module tick_gen and debounce_onepulse	20
3.4.3. Module sevenseg_driver	21
3.4.4. Top-Level Module Interface	21
3.4.5. Compile-Time Architectural Parameters	22
3.4.6. Symbol Encoding Block	22
3.4.7. Ladder Program Memory Arrays.....	23
3.4.8. Runtime State Registers.....	24
3.4.9. Edit-State Registers.....	25
3.4.10. Tick Generator Instantiation	25
3.4.11. Seven-Segment Driver Instantiation	26
3.4.12. Mode Selection Logic	26
3.4.13. Runtime Input Vector Construction.....	26
3.4.14. Source-Value Decode Function	27
3.4.15. Structural Validation Block	27

3.4.16.	Combinational Next-State Scan Block	28
3.4.17.	Lane Evaluation Sub-Block	28
3.4.18.	Action Execution Block.....	29
3.4.19.	Visible Rung Output Selection	29
3.4.20.	Display Formatting Block.....	30
3.4.21.	Reset Initialization Logic	30
3.4.22.	Run-Mode State Transfer	31
3.4.23.	TON Update Logic	31
3.4.24.	TOF Update Logic	32
3.4.25.	CTU Update Logic.....	32
3.4.26.	Built-In Demo Loader.....	32
3.4.27.	Debounce Module: debounce_onepulse.v	32
3.4.28.	Tick Generator Module: tick_gen.v	33
3.4.29.	Seven-Segment Driver Module: sevenseg_driver.v.....	33
3.4.30.	Constraint File Implementation	34
4.	Implementation	34
4.1.	HDL Modules and Their Functions	35
4.2.	Detailed Operating Principle of Edit and Run Modes	35
4.3.	Meaning of Every Edit Page	35
4.3.1.	Page 0: Coordinate Selection	35
4.3.2.	Page 1:Cell Programming.....	35
4.3.3.	Page 2: Action Programming.....	36
4.3.4.	Page 3: Preset Low Byte	36
4.3.5.	Page 4: Preset High Byte	36
4.3.6.	Page 5: Service/Validation Page.....	36
4.4.	Meaning of Inputs, Outputs LEDs and Display in Both Modes	37
4.5.	User Programming Workflow.....	37
4.6.	Worked Example Programs (Small, Medium, and Large).....	38
4.6.1.	Small Example: Single Normally-Open Contact	38
4.6.2.	Medium Example: Parallel Branches and Marker Latch	38
4.6.3.	Large Example: Four-Rung Integrated Demonstration.....	39
5.	Design Analysis and Evaluation	40
5.1.	Novelty.....	40
5.2.	Design Considerations (PO(c)).....	41
5.2.1.	Considerations to public health and safety	41

5.2.2. Considerations to environment	41
5.2.3. Considerations to cultural and societal needs	41
5.3. Investigations (PO(d)).....	41
5.3.1. Design of Experiment	41
5.3.2. Data Collection	42
5.3.3. Results and Analysis	42
5.3.4. Interpretation and Conclusions on Data.....	42
5.4. Limitations of Tools (PO(e))	43
5.5. Impact Assessment (PO(f)).....	43
5.5.1.Assessment of Societal and Cultural Issues	43
5.5.2.Assessment of Health and Safety Issues	43
5.5.3. Assessment of Legal Issues	43
5.6. Sustainability Evaluation (PO(g)).....	44
5.7. Ethical Issues (PO(h)).....	44
6. Reflection on Individual and Team work (PO(i))	44
6.1. Individual Contribution of Each Member.....	44
6.2. Mode of TeamWork.....	45
6.3. Diversity Statement of Team	45
6.4. Log Book of Project Implementation	46
7. Communication to External Stakeholders (PO(j))	47
7.1. Executive Summary	47
7.2. User Manual.....	47
7.3. YouTube Link.....	49
https://youtu.be/V50mvGif0Us?si=VbsnC_AaJEPqV7VM	49
8. Project Management and Cost Analysis (PO(k)).....	49
8.1. Bill of Materials	49
8.2. Timeline of Project Implementation	49
9. Future Work (PO(l)).....	50
10. References	51

1. Abstract

This report presents the design, implementation, and evaluation of an FPGA-based programmable logic controller (PLC) developed as a laboratory-scale educational platform. The controller is implemented in Verilog HDL on the Digilent Nexys A7-100T development board and executes ladder-logic programs entered directly through on-board switches and pushbuttons. The design supports up to eight rungs, three parallel lanes per rung, and seven logic cells per lane. Supported instruction classes include normally-open and normally-closed contacts, wire segments, output coils, set/reset instructions for outputs and markers, on-delay and off-delay timers, up counters, and counter reset actions. A dual-mode interface is used: Edit mode allows program entry through six page-based screens, while Run mode executes the ladder program and displays runtime state through LEDs and the eight-digit seven-segment display. The project demonstrates how industrial control concepts such as scan-cycle execution, rung evaluation, stateful marker latching, timer accumulation, and edge-triggered counting can be mapped to FPGA hardware. The report covers problem formulation, architectural design, scan algorithm, module-level implementation, user-interface design, test methodology, example ladder programs, and evaluation against educational, technical, and safety-oriented criteria. The result is a compact soft-PLC prototype suitable for teaching ladder semantics, digital design, and hardware realization of control logic.

2. Introduction

Programmable logic controllers are the dominant control platform in factory automation, process plants, building management systems, and conveyor-based industrial sequencing. The PLC became popular because it replaced large hard-wired relay panels with a programmable, maintainable, and more compact control platform. The most widely used graphical PLC language is ladder diagram (LD), which represents control logic using contact and coil symbols familiar to relay technicians.

In many undergraduate laboratories, ladder logic is discussed only at the conceptual level, or demonstrated using commercial PLC trainers that hide the actual hardware execution mechanism. As a result, students learn how ladder diagrams look, but not how they are represented internally, evaluated during a scan, or mapped into digital hardware.

FPGA-BASED PLC (Programmable Logic Controller) addresses that gap by implementing a small but functional soft PLC directly on an FPGA. Unlike a fixed demonstration circuit, this design can be reprogrammed live from the board itself. The same hardware platform is therefore able to demonstrate simple combinational ladder logic, latching behavior, timer-based delay, counter-based sequencing, and page-driven user interaction.

The project is also a complex engineering problem in the educational sense because it requires integration of multiple domains: digital logic design, synchronous state machine design, timing and debouncing, user-interface encoding, hardware resource management, and correct implementation of ladder semantics. Alternative approaches considered included a

microcontroller-based ladder interpreter, a PC-hosted simulator, and a precompiled FPGA demo. The FPGA route was selected because it offers deterministic hardware execution, direct visibility of internal state, and a strong connection to digital electronics coursework.

3. Design

3.1. Problem Formulation (PO(b))

The project objective was not merely to blink LEDs or implement a fixed sequence controller. The real challenge was to construct a compact, programmable controller that can accept ladder-like logic structures at runtime, preserve the semantics of industrial scan-based execution, and expose that behavior through limited on-board human interfaces. The design had to satisfy educational clarity, hardware realizability, and repeatable demonstration constraints at the same time.

3.1.1. Identification of Scope

The implemented scope is deliberately balanced between functionality and demonstration simplicity. The controller supports eight maximum rungs, three parallel lanes per rung, and seven logic cells per lane. This geometry is large enough to express useful multi-rung examples, yet small enough to fit comfortably in FPGA fabric and be edited through the board interface.

The logical source space includes 20 inputs, 16 outputs, 16 markers, eight on-delay timers, eight off-delay timers, and eight counters. The instruction set includes empty and wire cells, normally-open (NO) and normally-closed (NC) contacts, output coils, set/reset operations, timers, counters, and counter reset.

The project scope also includes a full user interaction mechanism. A single mode switch selects between Edit mode and Run mode. In Edit mode, the board itself acts as a compact programming panel. In Run mode, the same hardware becomes a real-time ladder execution platform.

The following items are intentionally outside the project scope: nonvolatile ladder storage, high-voltage or industrial field I/O, communication buses such as Modbus or Ethernet, a graphical HMI, and industrial safety certification.

Category	Included in This Project	Not Included / Future Extension
Programming model	On-board ladder entry; edit pages; rung/line/column addressing	PC-based graphical editor; import/export of ladder files
Execution	Runtime scan of ladder program; outputs, markers, timers, counters	Certified real-time industrial scheduler
Hardware I/O	Switches, buttons, LEDs, seven-segment display	Opto-isolated industrial inputs and relay outputs
Storage	Volatile register-based program/state storage	Persistent flash/EEPROM ladder storage

3.1.2. Literature Review

Industrial PLC practice is strongly associated with IEC 61131-3, which formalizes programming languages for programmable controllers, including ladder diagram (LD), function block diagram (FBD), structured text (ST), and sequential function chart (SFC) [1]. Ladder logic remains especially important because its contact-coil representation maps naturally to control circuits and relay replacements.

Educational and industrial texts on PLCs consistently emphasize the cyclic scan model: physical inputs are sampled, program logic is evaluated, and outputs are updated. This scan-cycle abstraction is central to correct reasoning about latching, edge-sensitive instructions, and timing behavior [2], [3].

Research on FPGA-based PLCs has shown that hardware realization can preserve ladder semantics while potentially increasing execution determinism and performance [4], [5]. At the same time, such systems must still address practical concerns such as timer resource usage, scan semantics, and user-oriented editing or compilation.

The Nexys A7 development board provides a suitable teaching platform because it combines an Artix-7 FPGA, a 100 MHz system clock, slide switches, pushbuttons, LEDs, and an eight-digit seven-segment display on a single ready-to-use board [6], [7]. AMD Vivado provides synthesis, implementation, and timing-analysis support for Verilog-based FPGA designs [8].

3.1.3. Formulation of Problem

The engineering problem can be stated as follows. Given a finite set of physical inputs, internal states, and output registers, design a synchronous digital system that stores a ladder-

like user program, evaluates that program correctly during each scan, and presents both configuration state and runtime state through a minimal set of board peripherals.

Let I denote the input vector, Q the output vector, M the internal marker vector, T the timer state set, and C the counter state set. For each rung r , each lane l , and each cell c , the design must compute the truth value of the cell source according to the programmed source class and index. It must then combine those values according to ladder semantics, where series cells are logically ANDed and non-empty lanes are logically ORed.

The design challenge is therefore not only one of combinational evaluation but also one of structured state management. Actions such as SET, RST, TON, TOF, and CTU introduce persistent or history-dependent behavior, which must be implemented using explicit registers and synchronous update logic.

The user-interface problem is equally important. Because only a limited number of switches and pushbuttons are available, the system must multiplex those controls across several edit pages while still remaining learnable and debuggable.

3.1.4. Analysis

At the logical level, each ladder cell is represented by three stored fields: cell type, source class, and source index. Cell type determines whether the cell is empty, a wire, a normally-open contact, or a normally-closed contact. Source class determines whether the referenced signal comes from the input table, output table, marker table, timer done table, off-delay timer table, or counter done table.

At the rung level, each lane is evaluated independently. If a lane contains at least one non-empty cell and all participating cells evaluate to true, then that lane is true. A rung becomes true if any non-empty lane is true. This is a direct hardware realization of the AND-OR structure expected from ladder diagrams.

At the scan level, rungs are executed sequentially from low index to high index. This top-to-bottom evaluation order matters because actions in an earlier rung can change Q or M values that are then read by later rungs within the same scan. This behavior closely matches the conceptual model used in PLC programming practice.

At the interface level, the main risk is operator confusion caused by reuse of the same switches on different edit pages. For example, a switch combination that selects a lane on Page 0 has a different meaning on Page 1, where it may instead define source class or source index. The documentation and user manual therefore need to enforce a careful page-by-page editing procedure.

3.2. Design Method (PO(a))

The design method combined formal ladder semantics, synchronous digital design principles, and practical FPGA implementation decisions. The first step was to define a compact internal

representation for ladder instructions. A memory-like register structure was chosen because the program must be editable during operation without resynthesizing the FPGA image.

The second step was to define the evaluation order and state-update policy. The controller reads physical inputs, computes rung truth values from the programmed cells, applies rung actions, and updates outputs and internal states synchronously. The design explicitly separates combinational scan evaluation from sequential state storage.

The third step was to build a board-only programming interface. Because a complete keyboard or graphical display was not available, a page-based configuration method was implemented. This method reuses the same switches for coordinate selection, cell entry, action entry, preset entry, and service commands.

3.2.1. Internal Program Representation

A register-array representation was selected for the ladder program. Each cell stores:

- (a) a 2-bit cell type,
- (b) a 3-bit source class,
- (c) a 5-bit source index.

Each rung stores:

- (a) a 4-bit action type,
- (b) a 4-bit action index,
- (c) a 16-bit action preset.

This allows the user to edit the ladder program dynamically without recompiling the FPGA.

Mathematical Model

For a cell value $x_{r,l,c}$, the lane truth is:

$$L_{r,l} = \prod x_{r,l,c}$$

using logical AND over all active cells in the lane.

The rung truth is:

$$R_r = L_{r,0} \vee L_{r,1} \vee L_{r,2}$$

using logical OR over all non-empty lanes.

This gives a direct mathematical interpretation of ladder diagrams:

- series = AND,
- parallel = OR.

3.2.2. High-Level Block Diagram (Data Path)

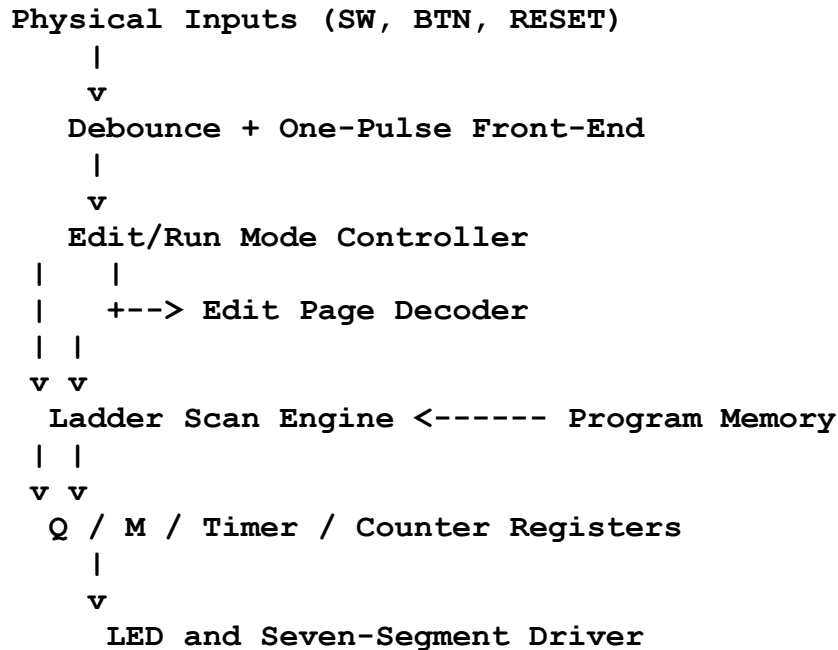


Fig. High-level functional data path of the FPGA-based PLC.

3.2.3. Scan-Cycle Algorithm

The ladder evaluation algorithm is implemented as a combinational scan that produces next-state control signals, followed by synchronous state updates on the system clock. This avoids uncontrolled asynchronous behavior while preserving PLC-like scan semantics.

1. Read current input vector I from the switches and pushbuttons.
2. Initialize temporary copies of Q, M, timer enable signals, counter enable signals, and rung status bits.
3. For each active rung, evaluate all lanes.
4. Within each lane, evaluate each non-empty cell according to cell type and source class.
5. AND all cells in a non-empty lane to obtain lane truth.
6. OR all true, non-empty lanes to obtain rung truth.
7. Execute the programmed action for the rung: coil, set/reset, timer, counter, or reset-counter.
8. Compute rung-visible status for LEDs and display.

9. At the next clock edge, store updated Q, M, timer/counter, and rung state values.

3.2.4. Timer and Counter Algorithms

- (i) The on-delay timer (TON) is enabled when its associated rung is true. While enabled, a 1 ms tick increments the accumulator until the programmed preset is reached. At or beyond the preset, the done bit becomes 1. When the enable becomes false, the accumulator and done bit are cleared.
- (ii) The off-delay timer (TOF) behaves in the complementary direction: while its rung is true, the done bit is asserted immediately. When the rung becomes false, a 1 ms accumulator is started. The done bit remains high during the timing interval and is cleared when the preset expires.
- (iii) The up counter (CTU) uses a rising-edge policy. A stored previous-enable bit is compared with the current enable. The accumulator increments only when current enable = 1 and previous enable = 0. The done bit is set when the accumulator reaches or exceeds the preset. A dedicated RESC action clears the accumulator and the done flag.

3.2.5. HDL Strategy

The design is partitioned into four Verilog modules:

1. plc_multi_rung_top.v (main controller),
2. debounce_onepulse.v (button conditioning),
3. tick_gen.v (1 ms and display refresh tick generation), and
4. sevenseg_driver.v (multiplexed display driver).

The top module contains both the edit-state machine and the scan engine. Arrays are used for per-cell and per-rung storage. Integer index functions are used to map the three-dimensional rung/lane/column space to linear storage addresses.

3.2.6. Simulation and Verification Strategy

This project is verified using progressive functional tests rather than a single large demonstration. The order is: (i) single NO contact, (ii) single NC contact, (iii) series AND, (iv) parallel OR, (v) latch and reset using markers, (vi) TON timer, (vii) CTU counter, and only then an integrated multi-rung example.

Because the board interface reuses switches across multiple edit pages, verification was included not only logic correctness but also operator workflow correctness. A common failure found was leaving a switch asserted when moving to another page, thereby unintentionally

programming an incorrect source class or source index

Test ID	Objective	Example Ladder	Expected Outcome
T1	Verify NO contact	[I0] --> (Q0)	Q0 follows I0
T2	Verify NC contact	[/I0] --> (Q0)	Q0 is high when I0 is low
T3	Verify series logic	[I0][I1] --> (Q0)	Q0 high only when both inputs are high
T4	Verify parallel logic	(I0 AND I1) OR (I2 AND /I3)	Either branch can make rung true
T5	Verify timer	M0 & I4 --> TON T0	T0.DN asserted after preset delay
T6	Verify counter	I8 --> CTU C0	Counter increments on rising edges only

3.2.7. Hardware Design and I/O Mapping

The hardware target is the Digilent Nexys A7-100T board, which provides the 100 MHz onboard clock and the user-facing peripherals required by the project [6], [7]. No custom PCB was designed because the purpose of the project was to develop a reusable teaching prototype on a standard FPGA trainer board.

The Verilog top module uses the following board resources: CLK100MHZ, CPU_RESETN, sixteen slide switches, five pushbuttons, sixteen LEDs, and the eight-digit seven-segment display. The XDC constraints file maps those signals to the Nexys A7 package pins.

The user interface is defined entirely in terms of digital board peripherals. This removes the need for external accessories, but it also demands careful documentation because the same controls are reused in both modes

Signal / Peripheral	Edit Mode Meaning	Run Mode Meaning	Remarks
SW15	Mode select	Mode select	0 = Edit mode, 1 = Run mode
SW0–SW14	Page-specific data entry	Inputs I0–I14	Meaning depends on current edit page when SW15=0
BTNL / BTNR	Previous / next page	Inputs I15 / I16	Debounced before use
BTNU / BTND	Column increment/decrement or delete	Inputs I17 / I18	BTND deletes a cell on Page 1
BTNC	Write / confirm	Input I19	Also service execution button on Page 5
LED[7:0]	Edit status bits	Rung visible results	Not always equal to raw Q outputs
LED[15:8]	Edit status bits	Inputs I0–I7	Shows selected low-order physical inputs in Run mode
7-seg display	E + page/coordinate/program data	A + runtime status data	Eight-digit multiplexed display

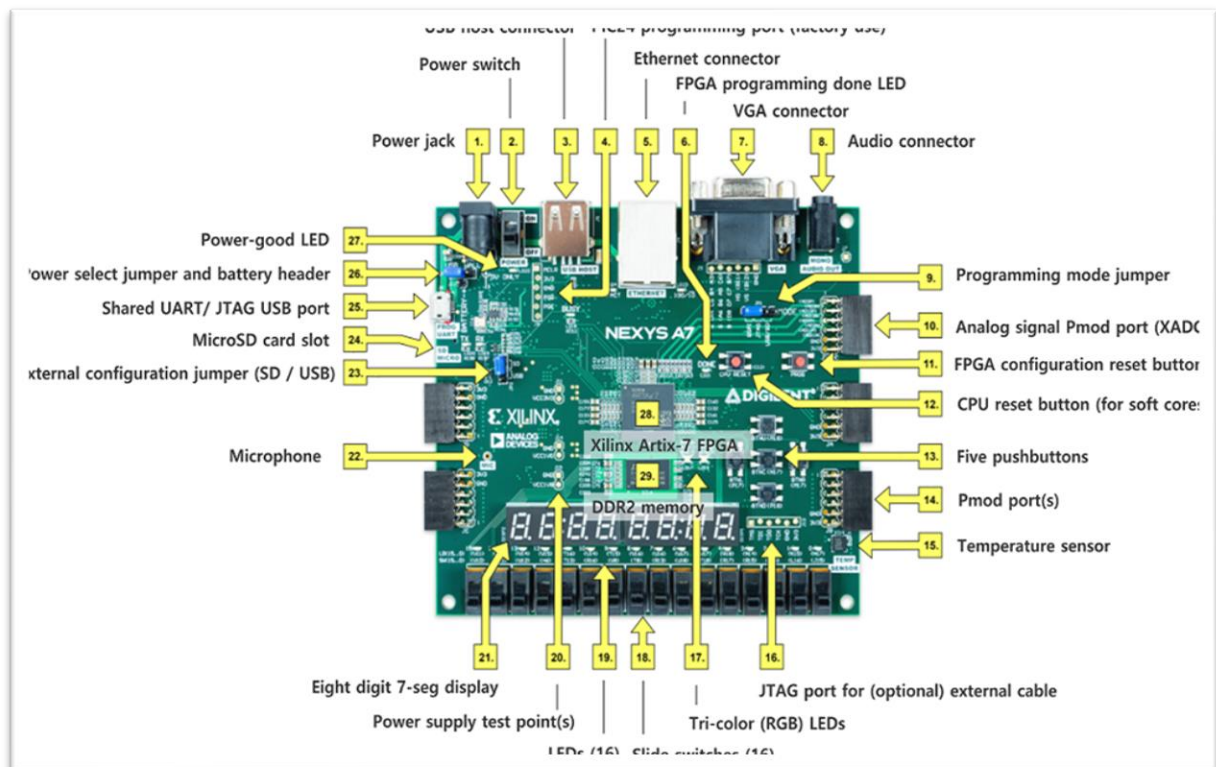


Fig: Hardware interface configuration diagram of the Nexys A7 board

3.3. Circuit Diagram (Simulated on Xilinx Vivado)

After simulation, The entire circuit was synthesized as 36 separate schematics, which created the whole architecture. Three of these schematics are attached below:

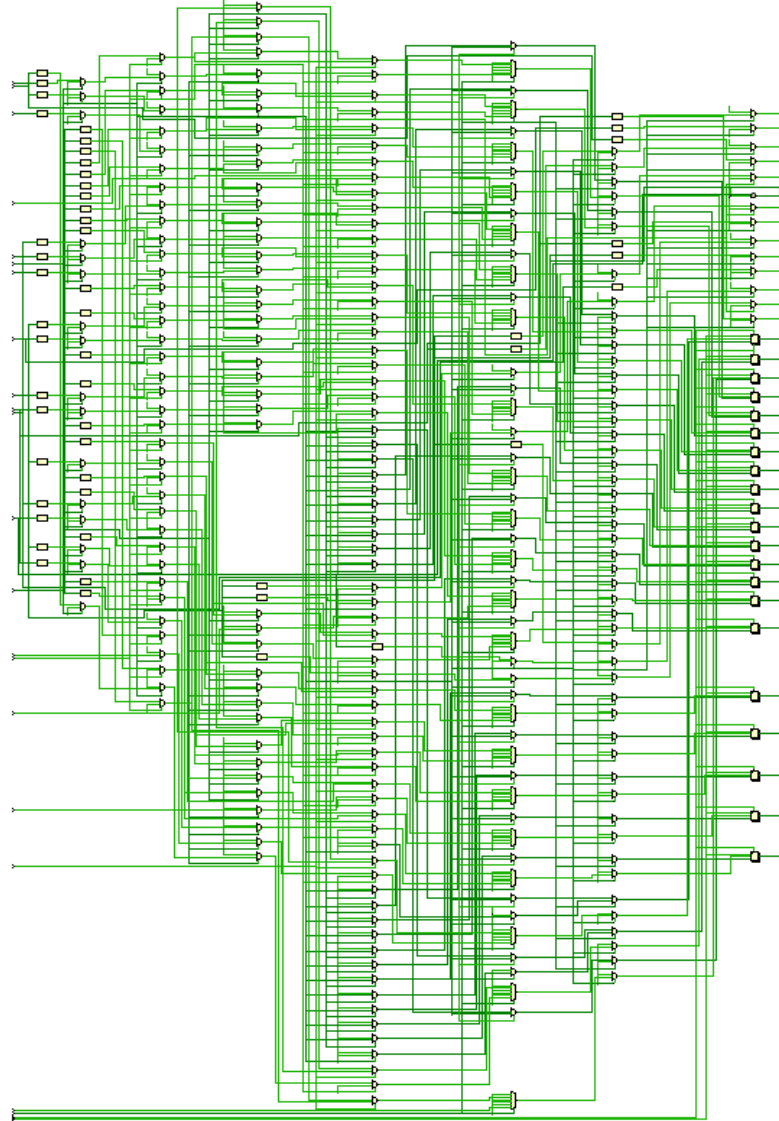


Fig1. HDL-Based Ladder Logic Scan Simulation

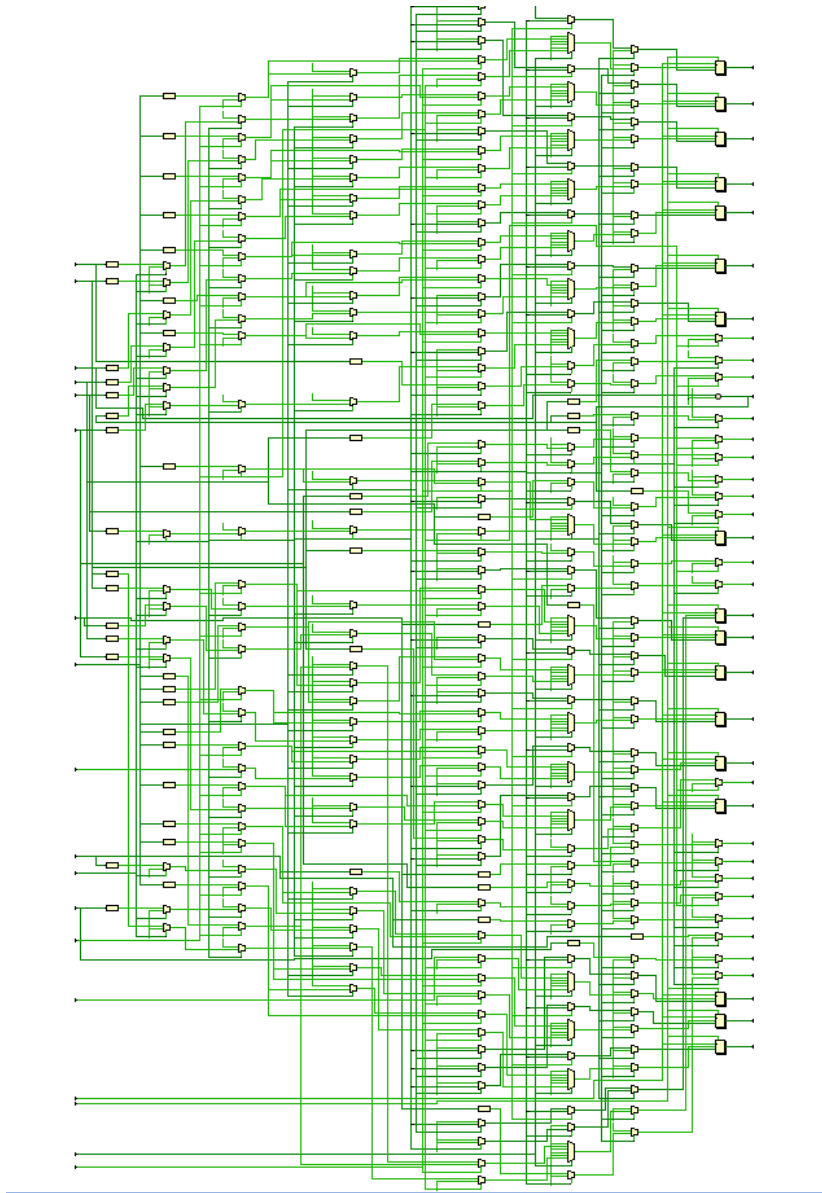


Fig .2: FPGA-Implemented PLC Execution Simulation

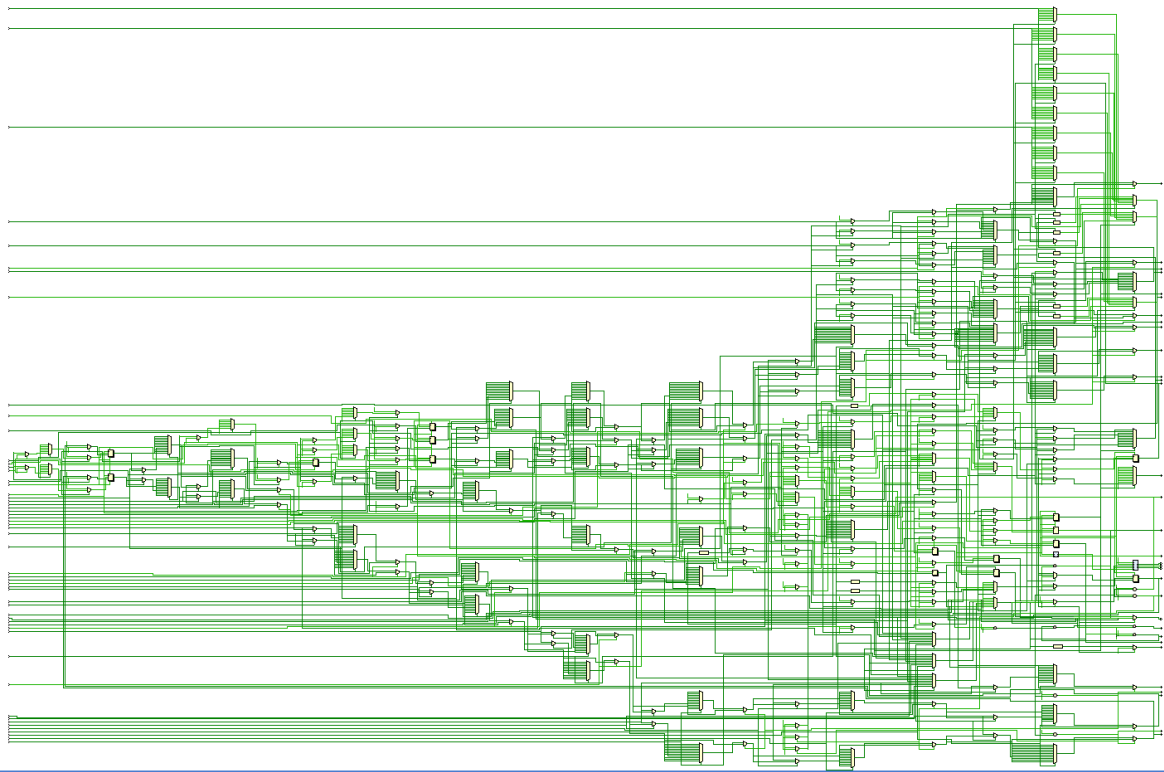


Fig.3: Multi-Rung Ladder Logic Behavioral Simulation

3.4. Full Source Code of Firmware

3.4.1. Top Module

<pre> module plc_multi_rung_top (input CLK100MHZ, input CPU_RESETN, input [15:0] sw, input BTNC, input BTNU, input BTND, input BTNL, input BTRN, output reg [15:0] LED, output [6:0] seg, output dp, output [7:0] an); localparam integer MAX_RUNGS = 8; localparam integer LANES = 3; localparam integer LOGIC_COLS = 7; localparam integer CELLS_PER_R = LANES * LOGIC_COLS; // 21 localparam integer TOTAL_CELLS = MAX_RUNGS * CELLS_PER_R; localparam integer N_INPUTS = 20; // sw[14:0] + 5 buttons localparam integer N_OUTPUTS = 16; localparam integer N_MARKERS = 16; localparam integer N_TIMERS = 8; localparam integer N_COUNTERS = 8; localparam [1:0] CT_EMPTY = 2'd0; localparam [1:0] CT_WIRE = 2'd1; localparam [1:0] CT_NO = 2'd2; localparam [1:0] CT_NC = 2'd3; localparam [2:0] SRC_NONE = 3'd0; localparam [2:0] SRC_I = 3'd1; localparam [2:0] SRC_Q = 3'd2; localparam [2:0] SRC_M = 3'd3; localparam [2:0] SRC_TONDN = 3'd4; localparam [2:0] SRC_TOFDN = 3'd5; localparam [2:0] SRC_CDN = 3'd6; localparam [3:0] ACT_NONE = 4'd0; localparam [3:0] ACT_COIL_Q = 4'd1; localparam [3:0] ACT_SET_Q = 4'd2; localparam [3:0] ACT_RST_Q = 4'd3; localparam [3:0] ACT_COIL_M = 4'd4; localparam [3:0] ACT_SET_M = 4'd5; localparam [3:0] ACT_RST_M = 4'd6; localparam [3:0] ACT_TON = 4'd7; localparam [3:0] ACT_TOF = 4'd8; localparam [3:0] ACT_CTU = 4'd9; localparam [3:0] ACT_RESC = 4'd10; reg [1:0] cell_type [0:TOTAL_CELLS-1]; reg [2:0] cell_src_class [0:TOTAL_CELLS-1]; reg [4:0] cell_src_idx [0:TOTAL_CELLS-1]; reg [3:0] rung_action_type [0:MAX_RUNGS-1]; reg [3:0] rung_action_idx [0:MAX_RUNGS-1]; reg [15:0] rung_action_pre [0:MAX_RUNGS-1]; reg [N_OUTPUTS-1:0] Q_table; reg [N_MARKERS-1:0] M_table; reg [15:0] TON_accum [0:N_TIMERS-1]; reg [15:0] TON_preset[0:N_TIMERS-1]; reg TON_dn [0:N_TIMERS-1]; reg TON_en [0:N_TIMERS-1]; reg [15:0] TOF_accum [0:N_TIMERS-1]; reg [15:0] TOF_preset[0:N_TIMERS-1]; </pre>	<pre> reg TOF_dn [0:N_TIMERS-1]; reg TOF_en [0:N_TIMERS-1]; reg [15:0] C_accum [0:N_COUNTERS-1]; reg [15:0] C_preset[0:N_COUNTERS-1]; reg C_dn [0:N_COUNTERS-1]; reg C_prev_en [0:N_COUNTERS-1]; reg [7:0] rung_true_reg; reg [7:0] rung_vis_reg; reg [2:0] edit_page; reg [2:0] cur_rung; reg [1:0] cur_lane; reg [2:0] cur_col; reg [2:0] active_rungs_m1; reg prog_dirty; reg prog_valid_reg; reg [7:0] error_code_reg; reg run_mode_d; wire reset_i; wire tick_1ms; wire tick_disp; wire btnc_level, btnu_level, btnd_level, btnl_level, btrn_level; wire btnc_pulse, btnu_pulse, btnd_pulse, btnl_pulse, btrn_pulse; wire run_mode; reg [N_INPUTS-1:0] I_run_vec; reg [N_OUTPUTS-1:0] q_scan_comb; reg [N_MARKERS-1:0] m_scan_comb; reg [N_TIMERS-1:0] ton_en_comb; reg [N_TIMERS-1:0] tof_en_comb; reg [N_COUNTERS-1:0] ctu_en_comb; reg [N_COUNTERS-1:0] resc_pulse_comb; reg [7:0] rung_true_comb; reg [7:0] rung_vis_comb; reg [15:0] ton_preset_next [0:N_TIMERS-1]; reg [15:0] tof_preset_next [0:N_TIMERS-1]; reg [15:0] c_preset_next [0:N_COUNTERS-1]; reg [31:0] disp_hex; reg [7:0] disp_dp; reg prog_valid_comb; reg [7:0] error_code_comb; integer val_r, val_l, val_c, val_idx; reg val_has_any; integer scan_r, scan_l, scan_c, scan_idx, scan_t; reg scan_srcbit; reg scan_lane_true; reg scan_lane_any; reg scan_rung_true_local; integer disp_idx; reg [1:0] disp_cur_type; reg [2:0] disp_cur_src_class; reg [4:0] disp_cur_src_idx; reg [3:0] disp_cur_act_type; reg [3:0] disp_cur_act_idx; reg [15:0] disp_cur_act_pre; reg [15:0] led_edit; integer seq_i, seq_r, seq_l, seq_c, seq_idx, seq_t; integer seq_cnext; assign reset_i = ~CPU_RESETN; </pre>
--	--

```

assign run_mode = sw[15];

function integer CIDX;
    input integer rung_id;
    input integer lane_id;
    input integer col_id;
    begin
        CIDX = (rung_id * CELLS_PER_R) + (lane_id *
        LOGIC_COLS) + col_id;
    end
endfunction

function src_value;
    input [2:0] src_class;
    input [4:0] src_idx;
    input [N_INPUTS-1:0] I_vec;
    input [N_OUTPUTS-1:0] Q_vec;
    input [N_MARKERS-1:0] M_vec;
    begin
        case (src_class)
            SRC_I:    src_value = I_vec[src_idx];
            SRC_Q:    src_value = Q_vec[src_idx];
            SRC_M:    src_value = M_vec[src_idx];
            SRC_TONDN: src_value = TON_dn[src_idx[2:0]];
            SRC_TOFDN: src_value = TOF_dn[src_idx[2:0]];
            SRC_CDN:  src_value = C_dn[src_idx[2:0]];
            default:  src_value = 1'b0;
        endcase
    end
endfunction

tick_gen u_tick_gen (
    .clk (CLK100MHZ),
    .reset (reset_i),
    .tick_1ms (tick_1ms),
    .tick_disp (tick_disp)
);

debounce_onepulse #(.COUNT_MAX(100000)) u_db_c (
    .clk(CLK100MHZ), .reset(reset_i), .din(BTNC),
    .level(btnc_level), .pulse(btnc_pulse)
);

debounce_onepulse #(.COUNT_MAX(100000)) u_db_u (
    .clk(CLK100MHZ), .reset(reset_i), .din(BTNU),
    .level(btnu_level), .pulse(btnu_pulse)
);

debounce_onepulse #(.COUNT_MAX(100000)) u_db_d (
    .clk(CLK100MHZ), .reset(reset_i), .din(BTND),
    .level(btnd_level), .pulse(btnd_pulse)
);

debounce_onepulse #(.COUNT_MAX(100000)) u_db_l (
    .clk(CLK100MHZ), .reset(reset_i), .din(BTNL),
    .level(btnl_level), .pulse(btnl_pulse)
);

debounce_onepulse #(.COUNT_MAX(100000)) u_db_r (
    .clk(CLK100MHZ), .reset(reset_i), .din(BTNR),
    .level(btnr_level), .pulse(btnr_pulse)
);

sevensseg_driver u_sevensseg (
    .clk (CLK100MHZ),
    .reset (reset_i),
    .tick (tick_disp),
    .hex_data (disp_hex),
    .dp_mask (disp_dp),
    .seg (seg),
    .dp (dp),
    .an (an)
);

always @(*) begin
    I_run_vec[14:0] = sw[14:0];
    I_run_vec[15] = btnl_level;
    I_run_vec[16] = btrn_level;
    I_run_vec[17] = btnu_level;
    I_run_vec[18] = btnd_level;

    I_run_vec[19] = btnc_level;
    end

    always @(*) begin
        prog_valid_comb = 1'b1;
        error_code_comb = 8'h00;

        for (val_r = 0; val_r < MAX_RUNGS; val_r = val_r +
        1) begin
            if (val_r <= active_rungs_m1) begin
                val_has_any = 1'b0;
                for (val_l = 0; val_l < LANES; val_l = val_l + 1)
                begin
                    for (val_c = 0; val_c < LOGIC_COLS; val_c = val_c +
                    1) begin
                        val_idx = CIDX(val_r, val_l, val_c);
                        if (cell_type[val_idx] != CT_EMPTY)
                            val_has_any = 1'b1;
                    end
                end

                if (!val_has_any && prog_valid_comb) begin
                    prog_valid_comb = 1'b0;
                    error_code_comb = 8'h01;
                end
                else if ((rung_action_type[val_r] == ACT_NONE) &&
                prog_valid_comb) begin
                    prog_valid_comb = 1'b0;
                    error_code_comb = 8'h02;
                end
            end
        end

        always @(*) begin
            for (scan_t = 0; scan_t < N_TIMERS; scan_t = scan_t
            + 1) begin
                ton_preset_next[scan_t] = TON_preset[scan_t];
                tof_preset_next[scan_t] = TOF_preset[scan_t];
            end

            for (scan_t = 0; scan_t < N_COUNTERS; scan_t =
            scan_t + 1) begin
                c_preset_next[scan_t] = C_preset[scan_t];
            end

            q_scan_comb = Q_table;
            m_scan_comb = M_table;
            ton_en_comb = {N_TIMERS{1'b0}};
            tof_en_comb = {N_TIMERS{1'b0}};
            ctu_en_comb = {N_COUNTERS{1'b0}};
            resc_pulse_comb = {N_COUNTERS{1'b0}};
            rung_true_comb = 8'h00;
            rung_vis_comb = 8'h00;

            for (scan_r = 0; scan_r < MAX_RUNGS; scan_r = scan_r
            + 1) begin
                if (scan_r <= active_rungs_m1) begin
                    scan_rung_true_local = 1'b0;

                    for (scan_l = 0; scan_l < LANES; scan_l = scan_l +
                    1) begin
                        scan_lane_true = 1'b1;
                        scan_lane_any = 1'b0;

                        for (scan_c = 0; scan_c < LOGIC_COLS; scan_c = scan_c
                        + 1) begin
                            scan_idx = CIDX(scan_r, scan_l, scan_c);
                            if (cell_type[scan_idx] != CT_EMPTY) begin
                                scan_lane_any = 1'b1;

                                case (cell_type[scan_idx])
                                    CT_WIRE: scan_srcbit = 1'b1;
                                    CT_NO: begin
                                        scan_srcbit = src_value(
                                        cell_src_class[scan_idx],
                                        cell_src_idx[scan_idx],
                                        I_run_vec,
                                        q_scan_comb,

```

<pre> m_scan_comb); end CT_NC: begin scan_srcbit = ~src_value(cell_src_class[scan_idx], cell_src_idx[scan_idx], I_run_vec, q_scan_comb, m_scan_comb); end default: scan_srcbit = 1'b1; endcase scan_lane_true = scan_lane_true & scan_srcbit; end end if (scan_lane_any && scan_lane_true) scan_rung_true_local = 1'b1; end rung_true_comb[scan_r] = scan_rung_true_local; case (rung_action_type[scan_r]) ACT_COIL_Q: begin if (rung_action_idx[scan_r] < N_OUTPUTS) q_scan_comb[rung_action_idx[scan_r]] = scan_rung_true_local; end ACT_SET_Q: begin if (scan_rung_true_local && (rung_action_idx[scan_r] < N_OUTPUTS)) q_scan_comb[rung_action_idx[scan_r]] = 1'b1; end ACT_RST_Q: begin if (scan_rung_true_local && (rung_action_idx[scan_r] < N_OUTPUTS)) q_scan_comb[rung_action_idx[scan_r]] = 1'b0; end ACT_COIL_M: begin if (rung_action_idx[scan_r] < N_MARKERS) m_scan_comb[rung_action_idx[scan_r]] = scan_rung_true_local; end ACT_SET_M: begin if (scan_rung_true_local && (rung_action_idx[scan_r] < N_MARKERS)) m_scan_comb[rung_action_idx[scan_r]] = 1'b1; end ACT_RST_M: begin if (scan_rung_true_local && (rung_action_idx[scan_r] < N_MARKERS)) m_scan_comb[rung_action_idx[scan_r]] = 1'b0; end ACT_TON: begin if (rung_action_idx[scan_r][2:0] < N_TIMERS) begin ton_en_comb[rung_action_idx[scan_r][2:0]] = scan_rung_true_local; ton_preset_next[rung_action_idx[scan_r][2:0]] = rung_action_pre[scan_r]; end end ACT_TOF: begin if (rung_action_idx[scan_r][2:0] < N_TIMERS) begin tof_en_comb[rung_action_idx[scan_r][2:0]] = scan_rung_true_local; tof_preset_next[rung_action_idx[scan_r][2:0]] = rung_action_pre[scan_r]; end end </pre>	<pre> ACT_CTU: begin if (rung_action_idx[scan_r][2:0] < N_COUNTERS) begin ctu_en_comb[rung_action_idx[scan_r][2:0]] = scan_rung_true_local; c_preset_next[rung_action_idx[scan_r][2:0]] = rung_action_pre[scan_r]; end end ACT_RESC: begin if (scan_rung_true_local && (rung_action_idx[scan_r][2:0] < N_COUNTERS)) resc_pulse_comb[rung_action_idx[scan_r][2:0]] = 1'b1; end default: begin end endcase case (rung_action_type[scan_r]) ACT_COIL_Q, ACT_SET_Q, ACT_RST_Q: begin if (rung_action_idx[scan_r] < N_OUTPUTS) rung_vis_comb[scan_r] = q_scan_comb[rung_action_idx[scan_r]]; else rung_vis_comb[scan_r] = scan_rung_true_local; end ACT_COIL_M, ACT_SET_M, ACT_RST_M: begin if (rung_action_idx[scan_r] < N_MARKERS) rung_vis_comb[scan_r] = m_scan_comb[rung_action_idx[scan_r]]; else rung_vis_comb[scan_r] = scan_rung_true_local; end ACT_TON: begin if (rung_action_idx[scan_r][2:0] < N_TIMERS) rung_vis_comb[scan_r] = TON_dn[rung_action_idx[scan_r][2:0]]; else rung_vis_comb[scan_r] = scan_rung_true_local; end ACT_TOF: begin if (rung_action_idx[scan_r][2:0] < N_TIMERS) rung_vis_comb[scan_r] = TOF_dn[rung_action_idx[scan_r][2:0]]; else rung_vis_comb[scan_r] = scan_rung_true_local; end ACT_CTU, ACT_RESC: begin if (rung_action_idx[scan_r][2:0] < N_COUNTERS) rung_vis_comb[scan_r] = C_dn[rung_action_idx[scan_r][2:0]]; else rung_vis_comb[scan_r] = scan_rung_true_local; end default: begin rung_vis_comb[scan_r] = scan_rung_true_local; end endcase end else begin rung_true_comb[scan_r] = 1'b0; rung_vis_comb[scan_r] = 1'b0; end end end always @(*) begin </pre>
---	--

<pre> disp_idx = CIDX(cur_rung, cur_lane, cur_col); disp_cur_type = cell_type[disp_idx]; disp_cur_src_class = cell_src_class[disp_idx]; disp_cur_src_idx = cell_src_idx[disp_idx]; disp_cur_act_type = rung_action_type[cur_rung]; disp_cur_act_idx = rung_action_idx[cur_rung]; disp_cur_act_pre = rung_action_pre[cur_rung]; led_edit = 16'h0000; led_edit[2:0] = cur_col; led_edit[4:3] = cur_lane; led_edit[7:5] = cur_rung; led_edit[9:8] = disp_cur_type; led_edit[12:10] = disp_cur_src_class; led_edit[13] = prog_valid_reg; led_edit[14] = prog_dirty; led_edit[15] = (error_code_reg != 8'h00); if (run_mode) begin LED[7:0] = rung_vis_reg; LED[15:8] = I_run_vec[7:0]; end else begin LED = led_edit; end if (run_mode) begin disp_hex[31:28] = 4'hA; disp_hex[27:24] = {1'b0, active_rungs_m1}; disp_hex[23:20] = rung_true_reg[3:0]; disp_hex[19:16] = rung_true_reg[7:4]; disp_hex[15:12] = rung_vis_reg[3:0]; disp_hex[11:8] = rung_vis_reg[7:4]; disp_hex[7:4] = Q_table[3:0]; disp_hex[3:0] = M_table[3:0]; disp_dp = 8'hFF; end else begin disp_hex[31:28] = 4'hE; disp_hex[27:24] = edit_page; disp_hex[23:20] = {1'b0, cur_rung}; disp_hex[19:16] = {2'b00, cur_lane}; disp_hex[15:12] = {1'b0, cur_col}; case (edit_page) 3'd0: begin disp_hex[11:8] = 4'h0; disp_hex[7:4] = 4'h0; disp_hex[3:0] = 4'h0; end 3'd1: begin disp_hex[11:8] = {2'b00, disp_cur_type}; disp_hex[7:4] = {1'b0, disp_cur_src_class}; disp_hex[3:0] = disp_cur_src_idx[3:0]; end 3'd2: begin disp_hex[11:8] = disp_cur_act_type; disp_hex[7:4] = disp_cur_act_idx; disp_hex[3:0] = 4'h0; end 3'd3: begin disp_hex[11:8] = disp_cur_act_pre[7:4]; disp_hex[7:4] = disp_cur_act_pre[3:0]; disp_hex[3:0] = 4'h0; end </pre>	<pre> 3'd4: begin disp_hex[11:8] = disp_cur_act_pre[15:12]; disp_hex[7:4] = disp_cur_act_pre[11:8]; disp_hex[3:0] = 4'h0; end default: begin disp_hex[11:8] = {1'b0, active_rungs_m1}; disp_hex[7:4] = error_code_reg[7:4]; disp_hex[3:0] = error_code_reg[3:0]; end endcase disp_dp = 8'hFF; end always @(posedge CLK100MHZ) begin if (reset_i) begin edit_page <= 3'd0; cur_rung <= 3'd0; cur_lane <= 2'd0; cur_col <= 3'd0; active_rungs_m1 <= 3'd0; prog_dirty <= 1'b0; prog_valid_reg <= 1'b0; error_code_reg <= 8'h00; run_mode_d <= 1'b0; Q_table <= {N_OUTPUTS{1'b0}}; M_table <= {N_MARKERS{1'b0}}; rung_true_reg <= 8'h00; rung_vis_reg <= 8'h00; for (seq_i = 0; seq_i < TOTAL_CELLS; seq_i = seq_i + 1) begin cell_type[seq_i] <= CT_EMPTY; cell_src_class[seq_i] <= SRC_NONE; cell_src_idx[seq_i] <= 5'd0; end for (seq_i = 0; seq_i < MAX_RUNGS; seq_i = seq_i + 1) begin rung_action_type[seq_i] <= ACT_NONE; rung_action_idx[seq_i] <= 4'd0; rung_action_pre[seq_i] <= 16'd0; end for (seq_i = 0; seq_i < N_TIMERS; seq_i = seq_i + 1) begin TON_accum[seq_i] <= 16'd0; TON_preset[seq_i] <= 16'd1000; TON_dn[seq_i] <= 1'b0; TON_en[seq_i] <= 1'b0; TOF_accum[seq_i] <= 16'd0; TOF_preset[seq_i] <= 16'd1000; TOF_dn[seq_i] <= 1'b0; TOF_en[seq_i] <= 1'b0; end for (seq_i = 0; seq_i < N_COUNTERS; seq_i = seq_i + 1) begin C_accum[seq_i] <= 16'd0; C_preset[seq_i] <= 16'd5; C_dn[seq_i] <= 1'b0; C_prev_en[seq_i] <= 1'b0; end active_rungs_m1 <= 3'd0; cell_type[CIDX(0,0,0)] <= CT_N0; cell_src_class[CIDX(0,0,0)] <= SRC_I; cell_src_idx[CIDX(0,0,0)] <= 5'd0; </pre>
--	---

<pre> cell_type[CIDX(0,0,1)] <= CT_NO; cell_src_class[CIDX(0,0,1)] <= SRC_I; cell_src_idx[CIDX(0,0,1)] <= 5'd1; rung_action_type[0] <= ACT_COIL_Q; rung_action_idx[0] <= 4'd0; end else begin run_mode_d <= run_mode; prog_valid_reg <= prog_valid_comb; error_code_reg <= error_code_comb; if (run_mode) begin Q_table <= q_scan_comb; M_table <= m_scan_comb; rung_true_reg <= rung_true_comb; rung_vis_reg <= rung_vis_comb; for (seq_t = 0; seq_t < N_TIMERS; seq_t = seq_t + 1) begin TON_preset[seq_t] <= ton_preset_next[seq_t]; TON_en[seq_t] <= ton_en_comb[seq_t]; if (!ton_en_comb[seq_t]) begin TON_accum[seq_t] <= 16'd0; TON_dn[seq_t] <= 1'b0; end else if (tick_1ms) begin if (TON_accum[seq_t] < ton_preset_next[seq_t]) TON_accum[seq_t] <= TON_accum[seq_t] + 16'd1; if ((TON_accum[seq_t] + 16'd1) >= ton_preset_next[seq_t]) TON_dn[seq_t] <= 1'b1; end TOF_preset[seq_t] <= tof_preset_next[seq_t]; TOF_en[seq_t] <= tof_en_comb[seq_t]; if (tof_en_comb[seq_t]) begin TOF_accum[seq_t] <= 16'd0; TOF_dn[seq_t] <= 1'b1; end else begin if (TOF_dn[seq_t]) begin if (tick_1ms) begin if (TOF_accum[seq_t] < tof_preset_next[seq_t]) TOF_accum[seq_t] <= TOF_accum[seq_t] + 16'd1; if ((TOF_accum[seq_t] + 16'd1) >= tof_preset_next[seq_t]) TOF_dn[seq_t] <= 1'b0; end end else begin TOF_accum[seq_t] <= 16'd0; end end end for (seq_t = 0; seq_t < N_COUNTERS; seq_t = seq_t + 1) begin C_preset[seq_t] <= c_preset_next[seq_t]; if (resc_pulse_comb[seq_t]) begin C_accum[seq_t] <= 16'd0; C_dn[seq_t] <= 1'b0; C_prev_en[seq_t] <= ctu_en_comb[seq_t]; end else begin seq_cnext = C_accum[seq_t]; if (ctu_en_comb[seq_t] && !C_prev_en[seq_t]) C_accum[seq_t] <= C_accum[seq_t] + 16'd1; C_prev_en[seq_t] <= ctu_en_comb[seq_t]; if (seq_cnext >= c_preset_next[seq_t]) C_dn[seq_t] <= 1'b1; else C_dn[seq_t] <= 1'b0; end end end else begin if (btnl_pulse) begin if (edit_page == 3'd0) edit_page <= 3'd5; else edit_page <= edit_page - 3'd1; end if (btnr_pulse) begin if (edit_page == 3'd5) edit_page <= 3'd0; else edit_page <= edit_page + 3'd1; end case (edit_page) 3'd0: begin if (btnc_pulse) begin cur_col <= sw[2:0]; cur_lane <= sw[4:3]; cur_rung <= sw[7:5]; end if (btnd_pulse) begin if (cur_col == 3'd6) cur_col <= 3'd0; else cur_col <= cur_col + 3'd1; end if (btnd_pulse) begin if (cur_col == 3'd0) cur_col <= 3'd6; else cur_col <= cur_col - 3'd1; end end 3'd1: begin seq_idx = CIDX(cur_rung, cur_lane, cur_col); if (btnc_pulse) begin cell_type[seq_idx] <= sw[1:0]; cell_src_class[seq_idx] <= sw[4:2]; cell_src_idx[seq_idx] <= sw[9:5]; prog_dirty <= 1'b1; end if (btnd_pulse) begin cell_type[seq_idx] <= CT_EMPTY; cell_src_class[seq_idx] <= SRC_NONE; cell_src_idx[seq_idx] <= 5'd0; prog_dirty <= 1'b1; end end 3'd2: begin </pre>	<pre> seq_cnext = C_accum[seq_t] + 16'd1; if (ctu_en_comb[seq_t] && !C_prev_en[seq_t]) C_accum[seq_t] <= C_accum[seq_t] + 16'd1; C_prev_en[seq_t] <= ctu_en_comb[seq_t]; if (seq_cnext >= c_preset_next[seq_t]) C_dn[seq_t] <= 1'b1; else C_dn[seq_t] <= 1'b0; end end end else begin if (btnl_pulse) begin if (edit_page == 3'd0) edit_page <= 3'd5; else edit_page <= edit_page - 3'd1; end if (btnr_pulse) begin if (edit_page == 3'd5) edit_page <= 3'd0; else edit_page <= edit_page + 3'd1; end case (edit_page) 3'd0: begin if (btnc_pulse) begin cur_col <= sw[2:0]; cur_lane <= sw[4:3]; cur_rung <= sw[7:5]; end if (btnd_pulse) begin if (cur_col == 3'd6) cur_col <= 3'd0; else cur_col <= cur_col + 3'd1; end if (btnd_pulse) begin if (cur_col == 3'd0) cur_col <= 3'd6; else cur_col <= cur_col - 3'd1; end end 3'd1: begin seq_idx = CIDX(cur_rung, cur_lane, cur_col); if (btnc_pulse) begin cell_type[seq_idx] <= sw[1:0]; cell_src_class[seq_idx] <= sw[4:2]; cell_src_idx[seq_idx] <= sw[9:5]; prog_dirty <= 1'b1; end if (btnd_pulse) begin cell_type[seq_idx] <= CT_EMPTY; cell_src_class[seq_idx] <= SRC_NONE; cell_src_idx[seq_idx] <= 5'd0; prog_dirty <= 1'b1; end end 3'd2: begin </pre>
--	---

<pre> if (btnc_pulse) begin rung_action_type[cur_rung] <= sw[3:0]; rung_action_idx[cur_rung] <= sw[7:4]; prog_dirty <= 1'b1; end end 3'd3: begin if (btnc_pulse) begin rung_action_pre[cur_rung][7:0] <= sw[7:0]; prog_dirty <= 1'b1; end end 3'd4: begin if (btnc_pulse) begin rung_action_pre[cur_rung][15:8] <= sw[7:0]; prog_dirty <= 1'b1; end end default: begin if (btnc_pulse) begin active_rungs_m1 <= sw[2:0]; if (sw[6]) begin for (seq_l = 0; seq_l < LANES; seq_l = seq_l + 1) begin for (seq_c = 0; seq_c < LOGIC_COLS; seq_c = seq_c + 1) begin seq_idx = CIDX(cur_rung, seq_l, seq_c); cell_type[seq_idx] <= CT_EMPTY; cell_src_class[seq_idx] <= SRC_NONE; cell_src_idx[seq_idx] <= 5'd0; end rung_action_type[cur_rung] <= ACT_NONE; rung_action_idx[cur_rung] <= 4'd0; rung_action_pre[cur_rung] <= 16'd0; prog_dirty <= 1'b1; end if (sw[4]) begin for (seq_r = 0; seq_r < MAX_RUNGS; seq_r = seq_r + 1) begin for (seq_l = 0; seq_l < LANES; seq_l = seq_l + 1) begin for (seq_c = 0; seq_c < LOGIC_COLS; seq_c = seq_c + 1) begin seq_idx = CIDX(seq_r, seq_l, seq_c); cell_type[seq_idx] <= CT_EMPTY; cell_src_class[seq_idx] <= SRC_NONE; cell_src_idx[seq_idx] <= 5'd0; end rung_action_type[seq_r] <= ACT_NONE; rung_action_idx[seq_r] <= 4'd0; rung_action_pre[seq_r] <= 16'd0; end prog_dirty <= 1'b1; end if (sw[5]) begin Q_table <= {N_OUTPUTS{1'b0}}; M_table <= {N_MARKERS{1'b0}}; rung_true_reg <= 8'h00; rung_vis_reg <= 8'h00; for (seq_t = 0; seq_t < N_TIMERS; seq_t = seq_t + 1) begin TON_accum[seq_t] <= 16'd0; TON_preset[seq_t] <= 16'd1000; TON_dn[seq_t] <= 1'b0; TON_en[seq_t] <= 1'b0; TOF_accum[seq_t] <= 16'd0; </pre>	<pre> TOF_preset[seq_t] <= 16'd1000; TOF_dn[seq_t] <= 1'b0; TOF_en[seq_t] <= 1'b0; end for (seq_t = 0; seq_t < N_COUNTERS; seq_t = seq_t + 1) begin C_accum[seq_t] <= 16'd0; C_preset[seq_t] <= 16'd5; C_dn[seq_t] <= 1'b0; C_prev_en[seq_t] <= 1'b0; end if (sw[3]) begin for (seq_r = 0; seq_r < MAX_RUNGS; seq_r = seq_r + 1) begin for (seq_l = 0; seq_l < LANES; seq_l = seq_l + 1) begin for (seq_c = 0; seq_c < LOGIC_COLS; seq_c = seq_c + 1) begin seq_idx = CIDX(seq_r, seq_l, seq_c); cell_type[seq_idx] <= CT_EMPTY; cell_src_class[seq_idx] <= SRC_NONE; cell_src_idx[seq_idx] <= 5'd0; end rung_action_type[seq_r] <= ACT_NONE; rung_action_idx[seq_r] <= 4'd0; rung_action_pre[seq_r] <= 16'd0; end active_rungs_m1 <= 3'd2; cell_type[CIDX(0,0,0)] <= CT_NO; cell_src_class[CIDX(0,0,0)] <= SRC_I; cell_src_idx[CIDX(0,0,0)] <= 5'd0; cell_type[CIDX(0,0,1)] <= CT_NO; cell_src_class[CIDX(0,0,1)] <= SRC_I; cell_src_idx[CIDX(0,0,1)] <= 5'd1; rung_action_type[0] <= ACT_COIL_Q; rung_action_idx[0] <= 4'd0; cell_type[CIDX(1,0,0)] <= CT_NO; cell_src_class[CIDX(1,0,0)] <= SRC_I; cell_src_idx[CIDX(1,0,0)] <= 5'd2; rung_action_type[1] <= ACT_SET_M; rung_action_idx[1] <= 4'd0; cell_type[CIDX(2,0,0)] <= CT_NC; cell_src_class[CIDX(2,0,0)] <= SRC_I; cell_src_idx[CIDX(2,0,0)] <= 5'd3; rung_action_type[2] <= ACT_RST_M; rung_action_idx[2] <= 4'd0; prog_dirty <= 1'b0; end if (btnd_pulse) prog_dirty <= 1'b0; end endcase end end end endmodule </pre>
--	--

Table: Source Code for the main program

3.4.2. Module tick_gen and debounce_onepulse

<pre> module tick_gen (input clk, input reset, output reg tick_1ms, output reg tick_disp); reg [16:0] cnt_1ms; reg [15:0] cnt_disp; always @(posedge clk) begin if (reset) begin cnt_1ms <= 17'd0; cnt_disp <= 16'd0; tick_1ms <= 1'b0; tick_disp <= 1'b0; end else begin tick_1ms <= 1'b0; tick_disp <= 1'b0; if (cnt_1ms == 17'd99999) begin cnt_1ms <= 17'd0; tick_1ms <= 1'b1; end else begin cnt_1ms <= cnt_1ms + 17'd1; end if (cnt_disp == 16'd4999) begin cnt_disp <= 16'd0; tick_disp <= 1'b1; end else begin cnt_disp <= cnt_disp + 16'd1; end end end endmodule </pre>	<pre> module debounce_onepulse #(parameter integer COUNT_MAX = 100000) (input clk, input reset, input din, output reg level, output reg pulse); reg sync0, sync1; reg [31:0] cnt; always @(posedge clk) begin if (reset) begin sync0 <= 1'b0; sync1 <= 1'b0; cnt <= 32'd0; level <= 1'b0; pulse <= 1'b0; end else begin sync0 <= din; sync1 <= sync0; pulse <= 1'b0; if (sync1 == level) begin cnt <= 32'd0; end else begin if (cnt == (COUNT_MAX-1)) begin cnt <= 32'd0; level <= sync1; if (sync1) pulse <= 1'b1; end else begin cnt <= cnt + 32'd1; end end end end endmodule </pre>
--	--

Table: Source Code for the sub modules

3.4.3. Module sevenseg_driver

<pre> module sevenseg_driver (input clk, input reset, input tick, input [31:0] hex_data, input [7:0] dp_mask, output reg [6:0] seg, output reg dp, output reg [7:0] an); reg [2:0] scan_sel; reg [3:0] digit; reg dp_on; always @(posedge clk) begin if (reset) scan_sel <= 3'd0; else if (tick) scan_sel <= scan_sel + 3'd1; end always @(*) begin case (scan_sel) 3'd0: begin digit = hex_data[3:0]; dp_on = dp_mask[0]; an = 8'b1111_1110; end 3'd1: begin digit = hex_data[7:4]; dp_on = dp_mask[1]; an = 8'b1111_1101; end 3'd2: begin digit = hex_data[11:8]; dp_on = dp_mask[2]; an = 8'b1111_1011; end 3'd3: begin digit = hex_data[15:12]; dp_on = dp_mask[3]; an = 8'b1111_0111; end 3'd4: begin digit = hex_data[19:16]; dp_on = dp_mask[4]; an = 8'b1110_1111; end endcase end end </pre>	<pre> 3'd5: begin digit = hex_data[23:20]; dp_on = dp_mask[5]; an = 8'b1101_1111; end 3'd6: begin digit = hex_data[27:24]; dp_on = dp_mask[6]; an = 8'b1011_1111; end default: begin digit = hex_data[31:28]; dp_on = dp_mask[7]; an = 8'b0111_1111; end endcase end always @(*) begin case (digit) 4'h0: seg = 7'b1000000; 4'h1: seg = 7'b1111001; 4'h2: seg = 7'b0100100; 4'h3: seg = 7'b0110000; 4'h4: seg = 7'b0011001; 4'h5: seg = 7'b0010010; 4'h6: seg = 7'b0000010; 4'h7: seg = 7'b1111000; 4'h8: seg = 7'b0000000; 4'h9: seg = 7'b0010000; 4'hA: seg = 7'b0001000; 4'hB: seg = 7'b0000011; 4'hC: seg = 7'b1000110; 4'hD: seg = 7'b0100001; 4'hE: seg = 7'b0000110; default: seg = 7'b0001110; endcase dp = dp_on; end endmodule </pre>
--	--

Table: Source Code for the sub modules

3.4.4. Top-Level Module Interface

The module `plc_multi_rung_top` defines the board-level I/O of the PLC core.

Inputs

- CLK100MHZ
- CPU_RESETN
- sw[15:0]
- BTNC, BTNU, BTND, BTNL, BTNR

Outputs

- LED[15:0]
- seg[6:0]
- dp
- an[7:0]

The top module integrates:

1. human input acquisition,
2. ladder program storage,
3. ladder scan evaluation,

4. runtime state update,
5. status formatting for LEDs and seven-segment display.

3.4.5. Compile-Time Architectural Parameters

At the beginning of the top module, several parameters define the logical size of the controller:

- ✓ MAX_RUNGS = 8
- ✓ LANES = 3
- ✓ LOGIC_COLS = 7
- ✓ N_INPUTS = 20
- ✓ N_OUTPUTS = 16
- ✓ N_MARKERS = 16
- ✓ N_TIMERS = 8
- ✓ N_COUNTERS = 8

Derived parameters are used to size the ladder-memory arrays and the loops inside the scan engine.

This parameterization determines the synthesized hardware capacity:

- 8 rungs maximum,
- 3 parallel branches per rung,
- 7 cells per branch,
- 20 source inputs,
- 16 outputs,
- 16 markers,
- 8 timers,
- 8 counters.

3.4.6. Symbol Encoding Block

The code declares `localparam` constants for all ladder symbols. These constants act as the internal instruction set of the PLC.

Cell type encoding

- (i) CT_EMPTY
- (ii) CT_WIRE
- (iii) CT_NO
- (iv) CT_NC

Source class encoding

- (i) SRC_NONE

- (ii) SRC_I
- (iii) SRC_Q
- (iv) SRC_M
- (v) SRC_TONDN
- (vi) SRC_TOFDN
- (vii) SRC_CDN

Action type encoding

- (i) ACT_NONE
- (ii) ACT_COIL_Q
- (iii) ACT_SET_Q
- (iv) ACT_RST_Q
- (v) ACT_COIL_M
- (vi) ACT_SET_M
- (vii) ACT_RST_M
- (viii) ACT_TON
- (ix) ACT_TOF
- (x) ACT_CTU
- (xi) ACT_RESC

This encoding allows the page editor to write compact instruction fields and the scan engine to decode them using simple case logic.

3.4.7. Ladder Program Memory Arrays

The ladder program is stored in register arrays rather than being hardwired as logic.

Cell storage arrays

For each cell:

- cell_type_mem
- cell_src_class_mem
- cell_src_idx_mem

These arrays represent the logical contact structure of the ladder.

Rung action arrays

For each rung:

- rung_action_type
- rung_action_idx
- rung_action_pre

These hold the instruction executed after rung truth is computed.

Storage organization

The ladder is conceptually indexed by:

- rung
- lane
- column

Internally, it is flattened into a linear memory space. This makes synthesis straightforward and allows the same indexing mechanism to be used in both edit mode and run mode.

3.4.8. Runtime State Registers

The PLC runtime state is held in dedicated register sets.

Output and marker state

- a. Q_table
- b. M_table

Timer state

For each timer:

- a. accumulator register,
- b. preset register,
- c. done bit,
- d. enable bit.

Separate state is maintained for TON and TOF behavior.

Counter state

For each counter:

- a. count register,
- b. preset register,
- c. done bit,
- d. previous enable bit.

The previous enable bit is necessary for edge detection in CTU.

Rung monitoring state

- `rung_true_reg`
- `rung_vis_reg`

These hold:

- raw logical result of each rung,
- user-visible result of each rung.

3.4.9. Edit-State Registers

The on-board editor is implemented using a small set of control registers:

- `edit_page`
- `cur_rung`
- `cur_lane`
- `cur_col`
- `active_rungs_m1`
- `prog_dirty`
- `prog_valid_reg`
- `error_code_reg`

These registers define:

- where the user is editing,
- how many rungs are currently active,
- whether the ladder has been modified,
- whether the ladder passes structural validation.

This block is effectively the control state of the HMI.

3.4.10. Tick Generator Instantiation

The `tick_gen` module is instantiated to derive lower-rate timing events from the 100 MHz board clock.

Generated pulses:

- `tick_1ms`
- `tick_disp`

These are used in:

- timer accumulation logic,
- display multiplex refresh logic.

The top module treats these tick signals as enable pulses rather than separate derived clocks, which is the preferred synchronous design style.

3.4.11. Seven-Segment Driver Instantiation

The `sevensseg_driver` module is instantiated with:

- a packed 32-bit display word,
- decimal-point mask,
- refresh tick.

The top module does not directly drive segment patterns. Instead, it builds the 32-bit logical display content and delegates the multiplexing to the driver module.

This keeps display formatting separate from display scanning.

3.4.12. Mode Selection Logic

The main mode signal is derived from `sw[15]`.

- `sw[15] = 0` → Edit mode
- `sw[15] = 1` → Run mode

The mode signal changes the interpretation of almost all user I/O:

- I) buttons become navigation/write controls in Edit mode,
- II) buttons become logic inputs in Run mode,
- III) display format changes,
- IV) LEDs change meaning,
- V) sequential logic follows either programming behavior or runtime PLC behavior.

3.4.13. Runtime Input Vector Construction

The code builds a 20-bit input vector for ladder evaluation.

Mapping:

- (1) `I_run_vec[14:0] = sw[14:0]`
- (2) `I_run_vec[15] = btnL_level`
- (3) `I_run_vec[16] = btnR_level`
- (4) `I_run_vec[17] = btnU_level`
- (5) `I_run_vec[18] = btnD_level`
- (6) `I_run_vec[19] = btnC_level`

This vector is the canonical PLC input table used by the source-value function.

Important detail:

- `sw[15]` is excluded because it is used for mode selection.

3.4.14. Source-Value Decode Function

The helper function `src_value()` returns the Boolean value of a ladder source.

Inputs to the function:

- i. source class,
- ii. source index,
- iii. runtime state tables.

Possible return sources:

- ❖ `I[index]`
- ❖ `Q[index]`
- ❖ `M[index]`
- ❖ `TON_dn[index]`
- ❖ `TOF_dn[index]`
- ❖ `C_dn[index]`

This function centralizes all source decoding into one place. As a result:

- ✓ NO/NC cell evaluation becomes simple,
- ✓ adding a new source class in future would require modification in only one function.

3.4.15. Structural Validation Block

A combinational validation block checks the stored program before or during editing.

For each active rung, it scans:

1. whether any cell in that rung is non-empty,
2. whether the rung action is non-zero.

Validation result:

- `prog_valid_reg`
- `error_code_reg`

Error code assignment:

- `00` = valid
- `01` = active rung has no programmed cell
- `02` = active rung has no action

This block is purely structural; it does not verify semantic quality, only minimum completeness.

3.4.16. **Combinational Next-State Scan Block**

The central combinational block computes the next scan result.

It first copies current state into temporary combinational variables:

- i) q_scan_comb
- ii) m_scan_comb
- iii) timer enable arrays
- iv) counter enable arrays
- v) reset pulse arrays
- vi) rung truth arrays
- vii) visible rung arrays

These signals represent the *candidate next state* after evaluating all active rungs.

This separation between:

- ✓ combinational next-state generation,
 - ✓ sequential register update,
- is the core hardware design method of the project.

3.4.17. **Lane Evaluation Sub-Block**

Inside the scan engine, each rung is processed lane by lane.

For each lane:

1. scan_lane_true is initialized to 1.
2. scan_lane_has_any is initialized to 0.
3. All cells in the lane are traversed.
4. For every non-empty cell:
 - a. scan_lane_has_any becomes 1,
 - b. the cell output is computed,
 - c. scan_lane_true is ANDed with the cell output.

Cell output generation:

- (a) WIRE → 1
- (b) NO → src_value
- (c) NC → ~src_value
- (d) EMPTY → skipped

Thus, the lane output equals the AND of all active cells.

3.4.18. Action Execution Block

After rung truth is known, the corresponding rung action is decoded.

Output actions

- COIL_Q: `q_scan_comb[idx] = rung_true`
- SET_Q: if rung true, set `q_scan_comb[idx]`
- RST_Q: if rung true, clear `q_scan_comb[idx]`

Marker actions

- COIL_M
- SET_M
- RST_M

These modify `m_scan_comb`.

Timer actions

- TON: sets `ton_en_comb[idx]` and writes preset
- TOF: sets `tof_en_comb[idx]` and writes preset

Counter actions

- CTU: sets `ctu_en_comb[idx]` and writes preset
- RESC: asserts `resc_pulse_comb[idx]`

This action block is effectively the instruction-execution stage of the PLC.

3.4.19. Visible Rung Output Selection

The visible rung result is not always identical to raw rung truth. The code therefore computes `rung_vis_comb[r]` separately.

Mapping depends on action type:

- coil/set/reset on Q → show resulting `Q[idx]`
- coil/set/reset on M → show resulting `M[idx]`
- TON → show `TON_dn[idx]`
- TOF → show `TOF_dn[idx]`
- CTU / RESC → show `C_dn[idx]`
- default → show raw rung truth

This logic explains why timer and counter rungs appear differently on the LED panel than a

purely combinational rung.

3.4.20. Display Formatting Block

A combinational formatting block constructs the 32-bit display word and LED output values.

Edit mode formatting

The display word contains:

- (i) E
- (ii) page index
- (iii) current rung
- (iv) current lane
- (v) current column
- (vi) page-specific payload

Payload depends on page:

- ✓ page 1 → cell type, source class, source index nibble
- ✓ page 2 → action type, action index
- ✓ page 3 → preset low byte
- ✓ page 4 → preset high byte
- ✓ page 5 → active rung count and error code

Run mode formatting

The display word contains:

- A
- active_rungs_m1
- lower and upper nibble of rung_true_reg
- lower and upper nibble of rung_vis_reg
- low nibble of Q_table
- low nibble of M_table

LED formatting:

- Edit mode LEDs show coordinate/type/status fields
- Run mode LEDs show rung_vis_reg[7:0] and I_run_vec[7:0]

3.4.21. Reset Initialization Logic

On reset, the sequential block performs full state initialization:

1. clears `Q_table`
2. clears `M_table`
3. clears rung truth and visible state
4. clears all cell memory
5. clears all rung actions
6. clears timer accumulators and done bits
7. clears counter accumulators and done bits
8. resets current page and coordinate
9. resets active-rung count

Then the code writes a default example program:

1. rung 0, lane 0, col 0 = NO I0
2. rung 0, lane 0, col 1 = NO I1
3. rung 0 action = COIL Q0

This serves as a power-on self-test configuration.

3.4.22. Run-Mode State Transfer

When the system is in Run mode, the sequential block copies the combinational next-state variables into registers:

- `Q_table <= q_scan_comb`
- `M_table <= m_scan_comb`
- `rung_true_reg <= rung_true_comb`
- `rung_vis_reg <= rung_vis_comb`

This is the clocked commit stage of the scan engine.

3.4.23. TON Update Logic

For each TON timer:

1. preset is updated from the programmed rung action,
2. enable is copied from `ton_en_comb`,
3. if enable is 0, accumulator and done are cleared,
4. if enable is 1 and `tick_1ms` occurs, accumulator increments,
5. when accumulator reaches preset, done bit becomes 1.

Thus the TON behaves as a synchronous up-counter driven by a 1 ms enable pulse.

3.4.24. TOF Update Logic

For each TOF timer:

1. preset is updated,
2. if enable is 1, done is asserted immediately and accumulator is cleared,
3. if enable is 0, the 1 ms tick increments the accumulator,
4. when accumulator reaches preset, done is deasserted.

This implements an off-delay response.

3.4.25. CTU Update Logic

For each counter:

1. preset is updated from the rung action,
2. if a reset pulse is present, count and done clear,
3. otherwise, current enable is compared with stored previous enable,
4. if a rising edge is detected, count increments,
5. if count reaches preset, done bit is asserted,
6. previous enable is updated.

This creates a synchronous edge-sensitive event counter.

3.4.26. Built-In Demo Loader

A specific Page 5 service action loads a predefined 3-rung demonstration program into ladder memory.

Implemented demo:

- rung 0: I0 AND I1 -> Q0
- rung 1: I2 -> SET M0
- rung 2: NC I3 -> RST M0

This is implemented as direct writes into the ladder memory arrays and rung action arrays inside the sequential control block.

3.4.27. Debounce Module: `debounce_onepulse.v`

This module is a synchronous digital filter for mechanical pushbuttons.

Internal structure

- two-stage synchronizer,
- debounce counter,
- stable output register,
- pulse-generation logic.

Operation

- input is first synchronized to the FPGA clock domain,
- if the synchronized input remains different from the stable state for long enough, the stable state is updated,
- when a new stable high state is accepted, a single-cycle pulse is generated.

This avoids metastability and multiple false events caused by switch bounce.

3.4.28. Tick Generator Module: tick_gen.v

This module is a clock-divider block.

It counts cycles of the 100 MHz clock and asserts:

- tick_1ms every 100,000 cycles,
- tick_disp at a faster periodic rate for display multiplexing.

This module does not generate secondary clocks; it generates synchronous enable pulses, which is preferable for FPGA design.

3.4.29. Seven-Segment Driver Module: sevenseg_driver.v

This module implements time-division multiplexing for the eight-digit seven-segment display.

Internal blocks

- scan selector counter,
- nibble multiplexer,
- hexadecimal-to-7-segment decoder,
- anode activation logic,
- decimal-point control logic.

At each display tick:

- the scan selector increments,

- one digit is activated,
- the corresponding 4-bit nibble is decoded into segment signals.

Because the scan frequency is high, all eight digits appear continuously illuminated.

3.4.30. Constraint File Implementation

The file `plc_multi_rung_top_xdc.xdc` binds the RTL ports to the physical FPGA pins corresponding to:

- clock source,
- reset,
- slide switches,
- pushbuttons,
- LEDs,
- seven-segment display.

Without this file, the design would synthesize but would not function correctly on the Nexys A7 board.

4. Implementation

The project was synthesized and implemented in Vivado. The placed design uses **13,577** slice LUTs (21.41%) and **2,894** slice registers (2.28%) of the target device according to the implementation report. The routed power report estimates total on-chip power at **0.170 W**. However, timing closure at 100 MHz is not achieved in the present design; the routed report shows a worst negative setup slack of **-46.578 ns**, so the prototype should be understood as a functional educational implementation rather than a timing-optimized production core.

The implementation is centered around the Verilog top module `plc_multi_rung_top.v`. This module instantiates the tick generator, five debounce/one-pulse modules, and the seven-segment display driver. It also contains the ladder program storage arrays, the runtime state tables, the program-validation logic, and the mode-dependent user interface.

On reset, the design initializes all program memory to empty and clears the runtime tables. A small default ladder is then loaded into rung 0: two normally-open contacts (I0 and I1) in series driving Q0 as a coil. This default behavior provides a quick functional check after configuration.

4.1. HDL Modules and Their Functions

Module	Role	Key Responsibilities
plc_multi_rung_top.v	Top-level PLC controller	Stores ladder program; handles edit pages; evaluates ladder; updates outputs, markers, timers, counters
debounce_onepulse.v	Button conditioning	Debounces noisy pushbuttons and generates single pulses for deterministic edits
tick_gen.v	Timebase generation	Generates 1 ms tick for timers and display refresh tick
sevenseg_driver.v	Display multiplexing	Drives the 8-digit seven-segment display from a 32-bit hex vector and decimal-point mask

4.2. Detailed Operating Principle of Edit and Run Modes

Mode selection is controlled by SW15. When SW15 = 0, the design enters Edit mode. The display shows the character E as the most significant digit, and the buttons are interpreted as navigation and commit controls rather than logic inputs.

When SW15 = 1, the design enters Run mode. The display shows the character A as the most significant digit, the buttons become logic inputs I15 through I19, and the ladder scan runs continuously.

A crucial implementation detail is that the same slide switches have different meanings on different edit pages. Consequently, the safe editing habit is: after every page change, first turn all switches OFF, then set only the switches needed for that page, and finally press BTNC.

4.3. Meaning of Every Edit Page

4.3.1. Page 0: Coordinate Selection

Purpose: select the current rung, lane, and column to be edited.

Switch fields: SW[2:0] = column (0–6), SW[4:3] = lane (0–2), SW[7:5] = rung (0–7).

Buttons: BTNC loads the coordinate from switches; BTNU increments column; BTND decrements column.

Display format: E | 0 | rung | lane | column | 0 | 0 | 0.

4.3.2. Page 1: Cell Programming

- i. *Purpose*: define the ladder cell at the selected coordinate.

- ii. *Switch fields:* SW[1:0] = cell type, SW[4:2] = source class, SW[9:5] = source index.
- iii. *Cell types:* 0 = EMPTY, 1 = WIRE, 2 = NO, 3 = NC.
- iv. *Source classes:* 0 = NONE, 1 = I, 2 = Q, 3 = M, 4 = TON.DN, 5 = TOF.DN, 6 = C.DN.
- v. *Buttons:* BTNC writes the cell; BTND deletes the current cell.
- vi. *Display format:* E | 1 | rung | lane | column | cell type | source class | source index (low nibble).

4.3.3. Page 2: Action Programming

- i. *Purpose:* define the action associated with the current rung.
- ii. *Switch fields:* SW[3:0] = action type, SW[7:4] = action index.
- iii. *Actions:* 0 = NONE, 1 = COIL Q, 2 = SET Q, 3 = RST Q, 4 = COIL M, 5 = SET M, 6 = RST M, 7 = TON, 8 = TOF, 9 = CTU, 10 = RESC
- iv. *Buttons:* BTNC writes the rung action.
- v. *Display format:* E | 2 | rung | lane | column | action type | action index | 0.

4.3.4. Page 3: Preset Low Byte

- i. *Purpose:* write bits [7:0] of the 16-bit preset field used by timers and counters.
- ii. *Switch field:* SW[7:0] = preset low byte.
- iii. *Button:* BTNC commits the low byte.
- iv. *Display format:* E | 3 | rung | lane | column | preset[7:4] | preset[3:0] | 0.

4.3.5. Page 4: Preset High Byte

- i. *Purpose:* write bits [15:8] of the 16-bit preset field.
- ii. *Switch field:* SW[7:0] = preset high byte.
- iii. *Button:* BTNC commits the high byte.
- iv. *Display format:* E | 4 | rung | lane | column | preset[15:12] | preset[11:8] | 0.

4.3.6. Page 5: Service/Validation Page

- i. *Purpose:* set active-rung count, load the demo program, clear program memory, clear runtime state, and inspect error code.
- ii. *Switch fields:* SW[2:0] = active_rungs_m1, SW3 = load demo, SW4 = clear all program, SW5 = clear runtime state only, SW6 = clear current rung only.
- iii. *Important detail:* active_rungs_m1 means number of active rungs minus 1. For example, to use 4 rungs, enter binary 011, not 100.
- iv. *Button:* BTNC executes the requested service actions.
- v. *Display format:* E | 5 | rung | lane | column | active_rungs_m1 | error_code[7:4] | error_code[3:0].
- vi. *Validation logic:* error 01 = active rung has no programmed cells; error 02 = active rung has no action

4.4. Meaning of Inputs, Outputs LEDs and Display in Both Modes

Run-mode logical inputs are formed as follows: I0–I14 come from SW0–SW14, while I15–I19 come from BTNL, BTNR, BTNU, BTND, and BTNC respectively. SW15 is reserved for mode selection and is not used as a ladder input.

The output table Q contains 16 bits (Q0–Q15), and the marker table M also contains 16 bits (M0–M15). Timers and counters each have eight instances. Timers include accumulator, preset, done bit, and enable state; counters include accumulator, preset, done bit, and previous-enable state.

In Edit mode, LEDs are reassigned as a compact status panel:

- a. LED[2:0] = current column,
- b. LED[4:3] = current lane,
- c. LED[7:5] = current rung,
- d. LED[9:8] = current cell type,
- e. LED[12:10] = current source class,
- f. LED13 = program-valid,
- g. LED14 = program-dirty,
- h. LED15 = error-present.

In Run mode, LED[7:0] show rung-visible results, not necessarily raw output bits. This distinction is critical: a timer rung, for instance, displays the timer done bit rather than simply showing the rung-input condition. LED[15:8] show physical inputs I0 through I7.

In Run mode, the seven-segment display uses the format

A | active_rungs_m1 | rung_true low nibble | rung_true high nibble | rung_vis low nibble | rung_vis high nibble | Q low nibble | M low nibble.

Thus the user can simultaneously observe whether each rung is logically true, whether the corresponding action is visibly active, and what the low-order output and marker bits currently are.

4.5. User Programming Workflow

- ✓ Set SW15 = 0 to enter Edit mode.
- ✓ Go to Page 5 and clear the previous program if required.
- ✓ On Page 5, set the number of active rungs minus one.
- ✓ Go to Page 0 and select the target rung, lane, and column.
- ✓ Go to Page 1 and write the cell type, source class, and source index.

- ✓ Repeat Page 0/Page 1 as needed to build the rung logic.
- ✓ Go to Page 2 and assign the rung action.
- ✓ If the action uses a preset (TON, TOF, or CTU), program the low byte on Page 3 and the high byte on Page 4.
- ✓ Return to Page 5, confirm the active-rung field, and check for validation errors.
- ✓ Set SW15 = 1 to enter Run mode and test the ladder using switches and pushbuttons.
- ✓ Best practice: before every programming step on Pages 0–5, first turn all switches OFF, then set only the switches needed for that page.

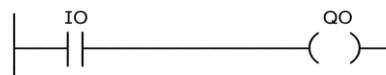
4.6. Worked Example Programs (Small, Medium, and Large)

The following examples are used as demonstration material and as a practical user guide. For each example, the safest operator habit is maintained: after each page transition, clear the switches first and then enter the new field values.

4.6.1. Small Example: Single Normally-Open Contact

Objective: implement the ladder [I0] -> (Q0).

Functional meaning: when SW0 is ON in Run mode, output Q0 becomes 1; when SW0 is OFF, Q0 becomes 0.



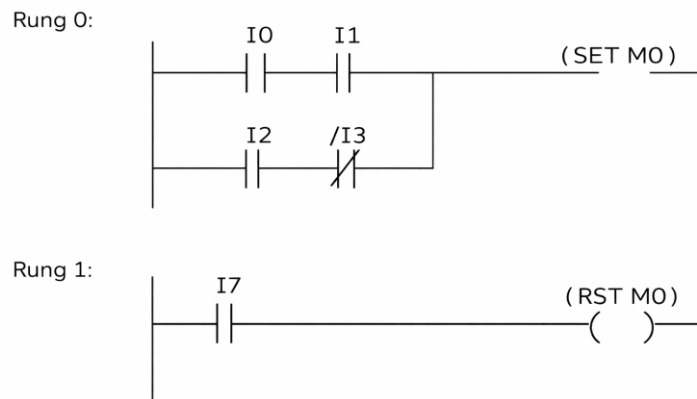
- i. Page 5: clear the program (SW4 only, then BTNC).
- ii. Page 5: set active_rungs_m1 = 0 because only rung 0 is used.
- iii. Page 0: select rung 0, lane 0, column 0 (all coordinate switches OFF, then BTNC).
- iv. Page 1: program cell type = NO, source class = I, source index = 0; press BTNC.
- v. Page 2: program action = COIL Q, action index = 0; press BTNC.
- vi. Page 5: confirm validation. LED13 should turn ON and LED15 should remain OFF.
- vii. Run mode: set SW15 = 1 and toggle SW0. Q0 should follow SW0. LED0 should represent rung-visible status, and the Q low nibble should appear in the second digit from the right on the display.

4.6.2. Medium Example: Parallel Branches and Marker Latch

Objective: set M0 using two alternative branches and reset it with a separate rung.

This example demonstrates series AND, parallel OR, NO and NC contacts, and marker

set/reset behavior.

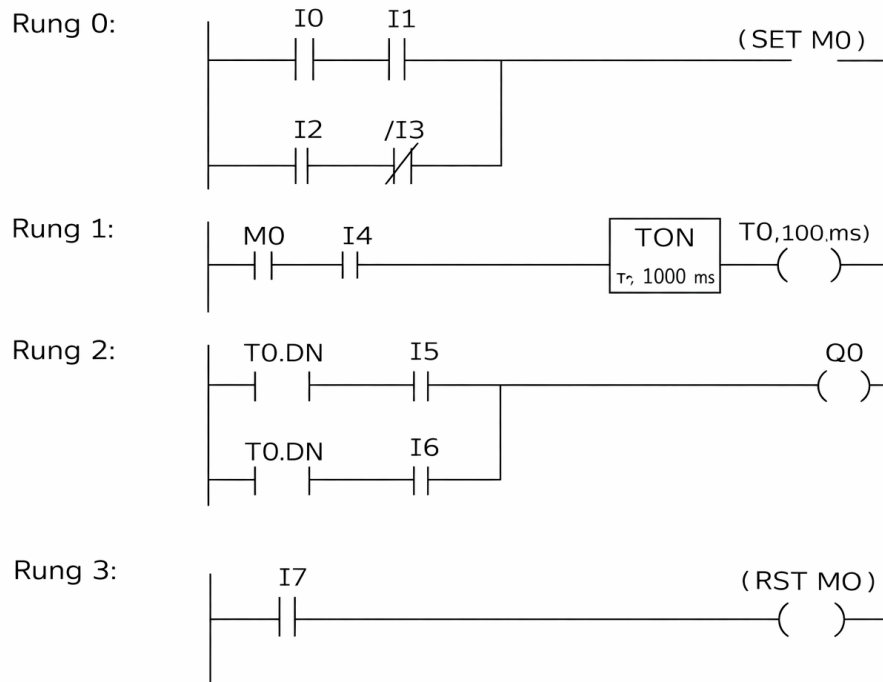


- i. Rung 0, lane 0, col 0: write NO I0.
- ii. Rung 0, lane 0, col 1: write NO I1.
- iii. Rung 0, lane 1, col 0: write NO I2.
- iv. Rung 0, lane 1, col 1: write NC I3.
- v. Rung 0 action: SET M, index 0.
- vi. Rung 1, lane 0, col 0: write NO I7.
- vii. Rung 1 action: RST M, index 0.
- viii. Validate the program on Page 5.
- ix. Run mode test 1: set SW0 = 1 and SW1 = 1; M0 should set.
- x. Run mode test 2: clear SW0 and SW1; M0 should remain latched.
- xi. Run mode test 3: set SW7 = 1; M0 should reset immediately.

4.6.3. Large Example: Four-Rung Integrated Demonstration

Objective: demonstrate multiple rungs, series logic, parallel branches, a marker latch, a timer, and output gating in one integrated ladder.

This is the final demonstration example because it is complex enough to show real PLC behavior while still remaining understandable to the audience.



- i. Page 5: clear the existing program and set active_rungs_m1 = 3.
- ii. Program Rung 0 as the two-branch SET M0 rung shown above.
- iii. Program Rung 1 as NO M0 in series with NO I4, action = TON T0.
- iv. On Page 3 and Page 4 of Rung 1, write preset 0x03E8, which equals 1000 decimal milliseconds.
- v. Program Rung 2 with two parallel branches: branch A = T0.DN AND I5, branch B = T0.DN AND I6, action = COIL Q0.
- vi. Program Rung 3 as NO I7 with action = RST M0.
- vii. Validate on Page 5; the error code should be 00 and LED13 should be ON.
- viii. Run mode test 1: set SW0 and SW1 high, or set SW2 high while SW3 remains low; M0 should set.
- ix. Run mode test 2: with M0 = 1, set SW4 high and wait about 1 second. The timer-done rung should become visible only after the delay.
- x. Run mode test 3: after T0.DN becomes 1, set SW5 or SW6 high. Q0 should turn on through either parallel branch.
- xi. Run mode test 4: set SW7 high. M0 should reset, the timer enable should drop, and the output path should collapse.

5. Design Analysis and Evaluation

5.1. Novelty

The novelty of this project is not that PLCs or FPGAs are new technologies. The novelty lies in combining them into a compact educational artifact that is both programmable and

hardware-visible. Unlike a precompiled FPGA demonstration, this system allows the user to build ladder logic live on the board. Unlike a software ladder simulator, it exposes real hardware constraints, timing, synchronous storage, and peripheral multiplexing.

The design is especially useful pedagogically because it reveals concepts that commercial PLCs usually hide: the difference between rung truth and rung-visible output, the meaning of internal markers, the effect of scan order, and the importance of edge-sensitive counting.

5.2. Design Considerations (PO(c))

5.2.1. Considerations to public health and safety

The prototype operates entirely at the logic level on a development board and does not switch mains power, motors, or industrial loads. This greatly reduces the electrical risk during classroom demonstration.

Because the design is not safety-certified and has no isolated field interface, it must not be connected directly to hazardous industrial equipment. The report and demonstration explicitly present it as an educational prototype only.

5.2.2. Considerations to environment

The project uses a reusable FPGA trainer board and does not require fabrication of a dedicated PCB, thereby reducing prototype waste.

Most debugging and iteration were performed in software and on the existing board, minimizing disposable materials and repeated hardware rework.

5.2.3. Considerations to cultural and societal needs

Industrial automation and PLC literacy remain important for electrical engineering graduates. This project helps bridge the gap between textbook ladder logic and hardware realization.

It is particularly suitable as a low-risk academic demonstrator that can support repeated teaching use across multiple student groups.

5.3. Investigations (PO(d))

5.3.1. Design of Experiment

The investigation plan used a progressive hierarchy of test cases. Each new feature was introduced only after the previous simpler case behaved correctly. This method reduced ambiguity and helped isolate interface errors from logic errors.

The principal experiments were: single-contact test, series-contact test, parallel-lane test,

marker latch test, timer-delay test, counter-edge test, and integrated multi-rung demonstration.

5.3.2. Data Collection

Data were collected in the form of observable board states rather than analog measurements: LED status, seven-segment patterns, and pass/fail behavior of expected rung outputs.

Vivado implementation reports were also collected as engineering data. These included resource utilization, timing summary, and estimated power.

5.3.3. Results and Analysis

Investigation	Observation	Interpretation	Status
NO/NC contact behavior	Matches expected truth table	Cell typing logic is correct	Pass
Series AND	Rung true only when all series contacts are true	Lane evaluation is correct	Pass
Parallel OR	Either branch can activate the rung when programmed carefully	Lane OR semantics are correct	Pass
Marker SET/RST	Marker latches and resets as expected	Internal-state register logic is correct	Pass
TON timer	Done bit asserts after preset delay and clears when enable drops	1 ms timing path is functionally correct	Pass
CTU counter	Count increases only on rising edges	Previous-enable edge detection works	Pass
100 MHz timing closure	Worst setup slack = -46.578 ns	Design is functionally demonstrable but not timing-closed at full clock	Needs optimization

5.3.4. Interpretation and Conclusions on Data

The functional investigations support the conclusion that the controller correctly implements the intended ladder semantics for the tested instruction set. Most user-facing problems encountered during development were not due to incorrect ladder evaluation but due to the compact page-based interface, especially switch reuse across pages.

The timing report reveals an important engineering limitation. Although the design behaves correctly in laboratory demonstration, it is not timing-closed at the board's 100 MHz system clock in its current form. This confirms that future optimization should focus on shortening

combinational paths, pipelining, or using slower derived scan clocks for the scan engine.

5.4. Limitations of Tools (PO(e))

The Nexys A7 board is an excellent teaching platform but not an industrial PLC backplane. Its switches and buttons are convenient for demonstration yet inefficient for large-scale ladder entry.

The current UI offers good transparency but limited ergonomic scalability. As program size grows, page-based entry becomes slower and more error-prone than a graphical ladder editor.

Vivado provides robust synthesis and implementation support, but the current design illustrates that a functionally correct RTL description can still fail timing closure if the combinational scan path becomes too deep.

The design stores the user program in FPGA registers only. Consequently, any power cycle, reset, or reprogramming event can erase the edited ladder unless it is re-entered.

Because no isolated industrial I/O stage is present, the prototype cannot safely interface directly with noisy or high-voltage field signals.

5.5. Impact Assessment (PO(f))

5.5.1. Assessment of Societal and Cultural Issues

The project supports engineering education by making automation concepts more accessible and concrete. It can be reused as a demonstrator in future laboratory offerings.

It also encourages local, low-cost educational innovation by showing that meaningful control-system prototypes can be developed on commonly available FPGA trainer boards.

5.5.2. Assessment of Health and Safety Issues

The controller is safe for low-voltage classroom use but should not be deployed for safety-critical automation without extensive redesign.

All demonstrations should be performed with logic-level loads only.

5.5.3. Assessment of Legal Issues

No industrial certification is claimed. The design must not be described as a safety PLC or a production-ready controller.

If the project were extended for wider distribution, compliance with EMC, electrical safety, and product-liability standards would become necessary.

5.6. Sustainability Evaluation (PO(g))

From an educational sustainability perspective, the project is strong. One reusable FPGA board can serve many cohorts and many example programs without producing repeated hardware waste.

From a product sustainability perspective, the Nexys trainer board is oversized and expensive compared with what a mass-produced educational micro-PLC would require. A future custom board could reduce material use, power, and cost significantly.

5.7. Ethical Issues (PO(h))

An important ethical responsibility in this project was honest representation. The system has educational value, but it is not an industrially qualified PLC. Overstating its readiness would be misleading to instructors, classmates, or external viewers.

Another ethical principle concerns attribution. Official board documentation, FPGA tool manuals, and PLC references were consulted and are cited in the references section. The report therefore distinguishes original implementation work from background knowledge obtained from external sources.

Team fairness is also an ethical issue. The project required multiple contributions-coding, debugging, testing, documentation, and presentation preparation-and these roles should be recorded transparently.

6. Reflection on Individual and Team work (PO(i))

6.1. Individual Contribution of Each Member

Student ID	Contribution
2106078	Contributed to the PLC core architecture, including ladder memory organization, rung/lane/cell structure, scan logic, and output/marker behavior. Also participated in integration testing and technical verification of the main control flow.
2106080	Led the project implementation and overall coordination. Took primary responsibility for top-level HDL integration, display/interface behavior, edit-mode page flow, and system-level debugging. Also played the main role in bringing together the ladder engine, user interface, board mapping, testing flow, and final demonstration readiness.
2106083	Contributed to the timing and support subsystem, including debounce logic, one-pulse generation, tick generation, and timer/counter verification. Also supported runtime debugging and preparation of test cases for dynamic ladder behavior.
2106084	Played a role in bringing together the ladder engine, user interface, board mapping, testing flow, and final demonstration readiness. Contributed to debounce logic, one-pulse generation, tick generation, and timer/counter verification. Also supported preparation of test cases for dynamic ladder

	behavior.
2106085	Contributed to user manual development, report writing, presentation preparation, and example-based validation. Also supported board-level testing, demonstration planning, and final formatting of the submission materials.

6.2. Mode of TeamWork

To foster effective teamwork, our project group adopted a collaborative strategy throughout the project. We held regular meetings to review progress, assign tasks, and resolve any challenges that arose. For seamless collaboration, especially during periods of remote work, we utilized various communication channels including email, messaging applications, and video conferences.

Each member had clearly defined roles and responsibilities, ensuring that tasks were distributed fairly based on individual skills and expertise. We leveraged collaborative platforms like Microsoft Onedrive for document sharing and GitHub for managing code repositories and project resources, which facilitated efficient teamwork and version control.

To ensure continuous improvement and high-quality deliverables, we cultivated an environment that encouraged open feedback and constructive criticism. Regular progress reports were presented during our meetings, allowing us to track project milestones and address any concerns or issues promptly.

6.3. Diversity Statement of Team

The project team, Group B1-05, benefited significantly from the diverse backgrounds, experiences, and perspectives of its members during the development of the FPGA-Based PLC (Programmable Logic Controller) project. Each member contributed distinct skills, technical knowledge, and ideas, which enhanced the overall quality of the work and supported the successful completion of the project.

The diversity within the team fostered a collaborative environment that encouraged innovation, critical thinking, and effective problem-solving. The different viewpoints of the members enabled a broader understanding of the project requirements and contributed to a more comprehensive evaluation of possible solutions. As a result, discussions were strengthened, decisions were made more effectively, and the overall outcomes of the project were improved.

Ultimately, the team recognized diversity as a valuable strength throughout the project. By combining the individual abilities and insights of its members, the group was able to address challenges efficiently and achieve the objectives of the FPGA-based PLC project successfully.

6.4. Log Book of Project Implementation

Date	Milestone achieved	Individual Role	Team Role	Comments
Week 3	Project Topic Assignment	All: went through the project topic	All members participated in topic discussion	Reviewed Scope
Week 6	PLC Theory Study and Literature Review	2106084: reviewed PLC scan-cycle behavior 2106083: studied NO/NC contacts, coils, latches 2106085: reviewed FPGA-based PLC implementation ideas	Team shared findings and finalized the target feature set	Literature review completed
Week 7	Overall Project Structure	2106080: designed top-level module structure 2106078: planned I/O mapping for switches, buttons etc	Team reviewed architecture and separated modules	Overall structure established
Week 8	Edit-Mode Interface Design	2106080: designed page-based programming flow 2106085: mapped button functions	User interaction scheme finalized	Edit-mode design completed
Week 9	Ladder Scan Engine Implementation	2106078: implemented rung action execution logic 2106083: verified small ladder examples and prepared truth-check cases	Team cross-checked series/parallel logic behavior	Core ladder engine developed successfully
Week 10	Timer, Counter, and Runtime State Integration	2106084: implemented TON/TOF timer logic 2106085: implemented CTU/RESC counter logic	Team validated timing and counter response	Stateful instruction support added
Week 11	Display, LED Mapping, and Debugging	2106078: integrated seven-segment display driver 2106080: finalized LED mapping for edit/run mode 2106083: debugged page-to-page switch errors	Team performed board-level debugging	Functional hardware demonstration achieved
Week 12	Testing and Validation	2106084: tested timer and counter examples All: prepared demo ladders	Team jointly validated all final demo cases	Testing phase completed
Week 13	Report Writing, Manual Preparation, and Slide Development	All: Performed peer evaluation, prepared slides, demo videos, poster and report	Team reviewed report, manual, and slides together	Documentation completed
Week 14	Final Presentation and Demonstration	All: Prepared slides and visualizations	Final presentation delivered	Project demonstrated successfully

7. Communication to External Stakeholders (PO(j))

7.1. Executive Summary

A group of BUET students has developed a compact FPGA-based programmable logic controller (PLC) that can be programmed directly from the board itself. Using switches, pushbuttons, LEDs, and a seven-segment display, the system allows users to enter ladder logic and instantly observe how it works in real time. The prototype demonstrates important PLC features such as contacts, coils, latches, timers, and counters in a simple, hands-on way. Although it is designed for education rather than industrial deployment, the project makes automation concepts easier to understand and shows how ladder logic can be implemented using digital hardware on an FPGA platform.

7.2. User Manual

The FPGA-Based PLC is a small educational programmable logic controller implemented on the Nexys A7 FPGA board. It allows the user to create and run ladder-logic programs directly from the board using switches, pushbuttons, LEDs, and the eight-digit seven-segment display. The system has two operating modes selected by SW15. When SW15 = 0, the controller is in Edit mode, where the ladder program is created or modified. When SW15 = 1, the controller is in Run mode, where the programmed ladder logic executes continuously in real time. The board reset button clears runtime state and reloads the default startup program.

In Run mode, the logical inputs are taken from the board controls. SW0 to SW14 act as PLC inputs I0 to I14, and the five pushbuttons act as I15 to I19. SW15 is reserved only for mode selection and is not used as a normal input. The controller also maintains internal outputs Q0 to Q15, markers M0 to M15, timer blocks, and counter blocks. These internal variables may be used as ladder sources or modified by rung actions. In Run mode, LED0 to LED7 show the visible result of the active rungs, while LED8 to LED15 show the low-order physical inputs I0 to I7. The seven-segment display begins with A and then shows the active-rung count minus one, rung truth information, visible rung status, and the low nibbles of Q and M. A very important point is that the rung LEDs do not always represent raw output bits; for timer or counter rungs, they may show the done status of that function block instead.

In Edit mode, the display begins with E, and the pushbuttons are used for programming. BTNL moves to the previous page, BTNR moves to the next page, BTNC confirms or writes the currently selected value, BTNU increases the column number on the coordinate-selection page, and BTND decreases the column number on that page or deletes the current cell on the cell-edit page. The editor is organized into six pages. Page 0 selects the current rung, lane, and column. On this page, SW[7:5] select the rung, SW[4:3] select the lane, and SW[2:0] select the column. After choosing the coordinate, BTNC stores it. Page 1 writes a ladder cell at that selected position. Here SW[1:0] choose the cell type, SW[4:2] choose the source class, and SW[9:5] choose the source index. The main cell types are EMPTY, WIRE, NO, and NC. The most common source classes are input I, output Q, marker M, timer done TON.DN, off-delay done TOF.DN, and counter done C.DN. Page 2 writes the action for the selected rung. On this page, SW[3:0] choose the action type and SW[7:4] choose the action index. Common actions are COIL Q, SET M, RST M, TON, TOF, CTU, and RESC. Pages 3 and 4

are used to enter the 16-bit preset value for timers and counters; Page 3 writes the low byte and Page 4 writes the high byte. Page 5 is the service and validation page. It is used to set the number of active rungs, clear a rung, clear the entire program, clear runtime state, or load the built-in demo. On this page, SW[2:0] represent active_rungs_m1, which means the number of active rungs minus one. Therefore, a value of 0 means 1 active rung, 1 means 2 active rungs, 2 means 3 active rungs, and so on. This is one of the most common sources of user error.

To create a program, first enter Edit mode by setting SW15 = 0. Go to Page 5 and clear the previous program if necessary. Then set the active-rung count minus one and confirm it with BTNC. Next, go to Page 0 and select the rung, lane, and column to be programmed. Move to Page 1 and enter the cell type, source class, and source index, then press BTNC to write the cell. Repeat this process for all required cells. After the cells of a rung are defined, go to Page 2 and enter the rung action and its index. If the rung uses a timer or counter, enter its preset value using Pages 3 and 4. When all programming is complete, return to Page 5 and check validation. In Edit mode, LED13 indicates that the program is valid, LED14 indicates that it has been modified, and LED15 indicates that an error exists. The error code shown on Page 5 identifies structural problems such as an active rung with no cells or an active rung with no action. Only after confirming valid status should the user switch to Run mode.

The ladder structure follows standard ladder semantics. Cells connected in series within a lane behave as logical AND. Different non-empty lanes within the same rung behave as logical OR. A normally open contact is true when its source is 1, and a normally closed contact is true when its source is 0. A coil writes the rung result to the selected destination. SET and RST latch or clear outputs and markers. TON produces an on-delay; the done bit becomes 1 only after the rung remains true for the preset duration. CTU counts rising edges of the rung result, and RESC resets the selected counter. Because the same switches are reused across multiple edit pages, the safest operating rule is to turn all switches OFF whenever you move to a new page, then set only the bits required for that page. This avoids accidental errors such as wrong source class, wrong source index, or incorrect parallel-lane programming.

For reliable operation, always verify the program before running it, keep track of the active-rung count carefully, and remember that timer and counter rungs may display their done status rather than simple input truth. If a rung behaves incorrectly, return to Edit mode, reselect the coordinate on Page 0, and rewrite the cell or action carefully. With correct page discipline, the controller can be used to demonstrate NO and NC contacts, series and parallel logic, markers, timers, counters, and complete multi-rung ladder programs in a fully hardware-based educational environment.

7.3. YouTube Link

https://youtu.be/V50mvGif0Us?si=VbsnC_AaJEPqV7VM

8. Project Management and Cost Analysis (PO(k))

8.1. Bill of Materials

Item	Quantity	Unit Cost	Total
Nexys A7-100T FPGA Trainer Board	1	USD 349.00	USD 349.00

As the FPGA board constituted an existing laboratory resource, the direct project costs were effectively zero.

8.2. Timeline of Project Implementation

Project Timeline											
Activity	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13
Project Assignment											
PLC Theory Study, Paper Study, and Planning											
Establish Overall Project Structure											
Testing and Debugging											
Documentation and Demo Preparation											
Final Presentation											

9. Future Work (PO(I))

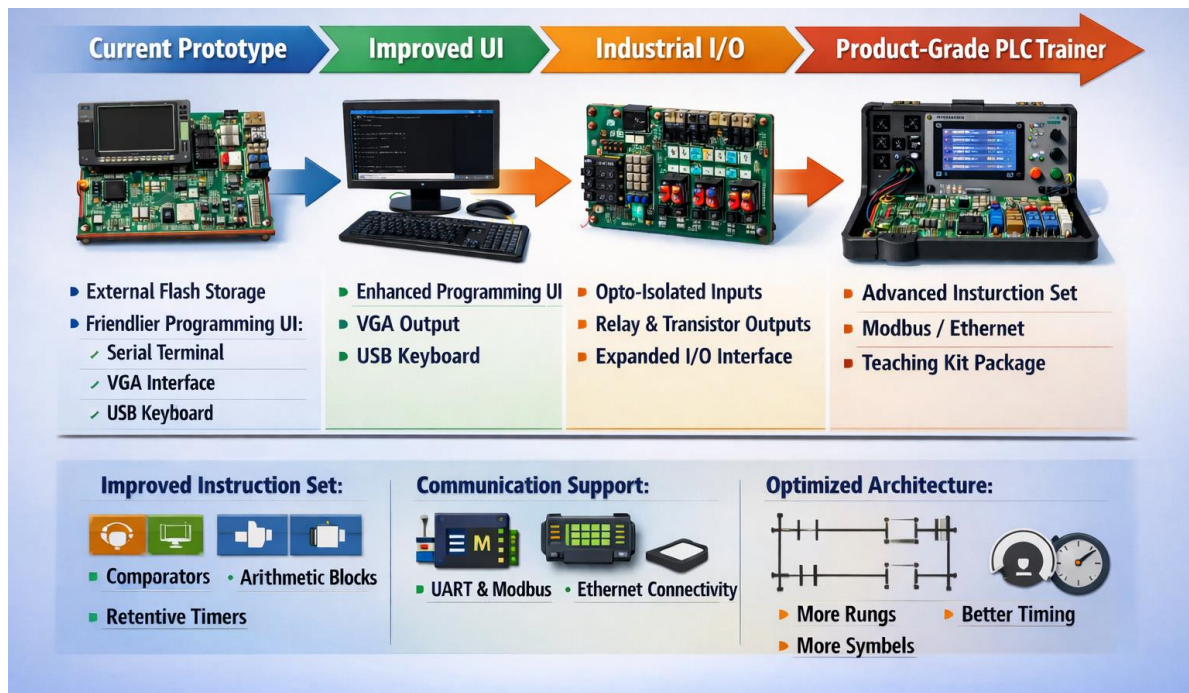


Fig: Future development roadmap of the FPGA-based PLC system

Although the proposed FPGA-based PLC prototype has successfully demonstrated ladder logic execution and on-board programming, several improvements may be undertaken to enhance its practicality, functionality, and suitability for real-world applications. The future development of the system may be considered as a progressive roadmap consisting of the following stages.

❑ Enhancement of the Current Prototype

At present, the system operates using the peripherals available on the FPGA board and stores ladder programs in volatile memory. An immediate area for improvement is the incorporation of external Flash memory to provide nonvolatile program storage. This would allow user-defined ladder programs to be retained after power loss, thereby improving system reliability and convenience during repeated operation. Furthermore, the existing switch-based programming mechanism may be refined to reduce operational complexity and improve overall usability.

❑ Improved Programming Interface

A subsequent stage of development involves the introduction of a more efficient and user-friendly programming interface. Rather than relying exclusively on switches and pushbuttons, future implementations may integrate a serial terminal for text-based ladder programming, VGA output for displaying ladder diagrams and system states, and USB keyboard support for direct user input. These additions would facilitate easier program creation, modification, and debugging. Moreover, a graphical or semi-graphical interface would enable real-time visualization of active rungs, contacts, and outputs, thereby significantly improving the quality of user interaction.

❑ Industrial I/O Implementation

To extend the prototype toward practical industrial use, the system should incorporate industrial-grade input and output interfaces. Such implementation may include opto-isolated inputs for the safe connection of external sensors, relay- or transistor-based outputs for actuator control, and expandable I/O modules to support larger control systems. These enhancements would provide improved electrical isolation, greater noise immunity, and safer operation in industrial environments, thereby enabling the FPGA-based PLC to interface effectively with real devices such as motors, switches, and control panels.

❑ Development of a Product-Grade PLC Trainer

In the final stage, the prototype may be further developed into a complete product-grade PLC trainer or controller. This would require several important advancements. First, the instruction set may be expanded to include comparators, arithmetic functions, edge-triggered contacts, and retentive timers, thereby bringing the system closer to the functional capabilities of commercial PLCs. Second, communication features such as UART-based diagnostics, Modbus support, and Ethernet connectivity may be incorporated to enable debugging, monitoring, and integration with industrial supervisory systems. Third, the overall architecture may be optimized to support a larger number of rungs, symbols, and instructions while improving timing performance and FPGA resource utilization. Finally, the entire system may be packaged as a dedicated educational training kit with integrated display modules, input interfaces, and software tools, making it suitable for laboratory instruction in digital electronics and industrial automation.

10. References

- [1] IEC, *IEC 61131-3:2025 Programmable Controllers – Part 3: Programming Languages*, International Electrotechnical Commission, 2025.
- [2] F. D. Petruzella, *Programmable Logic Controllers*, 5th ed. McGraw-Hill, 2016.
- [3] W. Bolton, *Programmable Logic Controllers*, 6th ed. Newnes/Elsevier, 2015.
- [4] H. Zhu, B. Liang, B. Dong, and X. Feng, “Research on FPGA-based programmable logic controllers’ technology,” *TELKOMNIKA*, vol. 11, no. 12, pp. 7655–7663, 2013.
- [5] N. W. Bergmann, “FPGA implementations of ladder diagrams,” *Modern Applied Science*, vol. 7, no. 3, pp. 64–80, 2013.
- [6] Digilent, “Nexys A7 Reference Manual” Digilent Reference.
<https://digilent.com/reference/programmable-logic/nexys-a7/reference-manual>
- [7] Digilent, “Nexys A7 AMD Artix-7 FPGA Trainer Board: Recommended for ECE Curriculum.”
- [8] AMD, “Vivado Design Suite User Guide: Synthesis (UG901).”

- [9] AMD, “7 Series FPGAs Clocking Resources User Guide (UG472).”
- [10] Department of EEE, BUET, “Program Educational Objectives and Program Outcomes.”
- [11] AMD, "7 Series FPGAs Clocking Resources User Guide (UG472)."https://docs.amd.com/v/u/en-US/ug472_7Series_Clocking
- [12] Implementation of PLC using FPGA -
<https://www.ijser.in/archives/v4i10/IJSER151015.pdf>
- [13] Introduction to Logic Circuits & Logic Design with Verilog by B. J. LaMeres -
<https://link.springer.com/book/10.1007/978-3-030-13605-5>